**An Adaptive Skill-Based Learning Path Recommendation System**

A Major Project report submitted

in partial fulfillment of requirement for the award of degree

**BACHELOR OF TECHNOLOGY**

in

**SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE**

by


| | |
|---|---|
| **A. Jasper Lalitha Enoch** | **2203A51535** |
| **K. Laxmi Prasanna** | **2203A51423** |
| **K. Amrutha** | **2203A51747** |
| **G. Varshitha** | **2203A51675** |
| **A. Nikhitha Sri** | **2203A51534** |
| **R. Amulya** | **2203A51059** |

Under the guidance of

**Dr. Durgesh Nandan**

Associate Professor, School of CS&AI.

SR UNIVERSITY

SR University, Ananthsagar,Warangal,Telagnana-506371

# SR University

Ananthasagar, Warangal.

SR UNIVERSITY

## CERTIFICATE

This is to certify that this project entitled **"An Adaptive Skill-Based Learning Path Recommendation System"** is the bonafied work carried out by **A. Jasper Lalitha Enoch, K. Laxmi Prasanna, K. Amrutha, G. Varshitha, A. Nikhitha Sri, R. Amulya** as a Major Project for the partial fulfillment to award the degree **BACHELOR OF TECHNOLOGY** in **School of Computer Science and Artificial Intelligence** during the academic year 2025-2026 under our guidance and Supervision.

**Dr. Durgesh Nandan**
Associate Professor,
SR University
Anathasagar,Warangal

**Dr. CH. Sandeep**,
Associate Professor & Head,
School of CS&AI,
SR University
Ananthasagar, Warangal.

**Reviewer-1**
Name:
Designation:
Signature:

**Reviewer-2**
Name:
Designation:
Signature:

# ACKNOWLEDGEMENT

We owe an enormous debt of gratitude to our Major Project guide **Dr. Durgesh Nandan, Associate Professor** as well as Head of the School of CS&AI , **Dr. CH.Sandeep, Associate Professor** and Dean of the School of CS&AI, **Dr.M. Sheshikala, Professor** for guiding us from the beginning through the end of the Major Project with their intellectual advices and insightful suggestions. We truly value their consistent feedback on our progress, which was always constructive and encouraging and ultimately drove us to the right direction.

We express our thanks to project co-ordinators **Mr. Sallauddin Md, Asst. Prof.,** for their encouragement and support.

Finally, we express our thanks to all the teaching and non-teaching staff of the department for their suggestions and timely support.

**Table of Contents**

**CHAPTER 1**.....3

1.1 INTRODUCTION.....3

**CHAPTER 2**.....6

2.1 RELATED WORK .....6

**CHAPTER 3**.....12

3.1 PROBLEM STATEMENT .....12

**CHAPTER 4**.....14

4.1 REQUIREMENT ANALYSIS.....14

4.2 RISK ANALYSIS .....16

4.3 FEASIBILITY ANALYSIS.....18

**CHAPTER 5**.....22

5.1 PROPOSED SOLUTION .....22

**CHAPTER 6**.....24

6.1 SYSTEM ARCHITECTURE.....24

6.2 DATA FLOW DIAGRAM .....28

6.3 FLOW CHART.....30

6.4 UML DIAGRAMS.....33

6.4.1 COMPONENT DIAGRAM .....33

6.4.2 ACTIVITY DIAGRAM.....34

6.4.3 STATE MACHINE DIAGRAM .....35

6.4.4 USECASE DIAGRAM .....36

**CHAPTER 7**.....37

7.1 SYSTEM SETUP AND IMPLEMENTATION .....37

**CHAPTER 8**.....41

8.1 METHODOLOGY .....41

8.2 RESULT AND ANALYSIS .....43

8.3 LEARNING OUTCOMES.....44

**CHAPTER 9**.....46

9.1 CONCLUSION.....46

**CHAPTER 10**.....49

10.1 REFERENCES.....49

## ABSTRACT

With the rapid growth of online learning platforms, students are exposed to vast amounts of educational content, making it increasingly difficult to identify what to learn next. Traditional recommendation systems often rely on simplistic approaches such as popularity-based filtering or basic collaborative methods, which fail to capture the complex relationships between learners, questions, and underlying concepts. This results in less personalized and less effective learning experiences.

This project presents an intelligent **Learning Path Recommendation System using Graph Neural Networks (GNNs)** that models the learning process as a graph structure. In this system, users, questions, and skills are represented as nodes, while their interactions form edges, enabling the model to capture deeper relationships and dependencies within the learning ecosystem.

To enhance recommendation performance, multiple GNN models including GCN, GAT, GraphSAGE, GIN, and RGCN are utilized and combined through an ensemble approach. Additionally, a temporal component using a Temporal Graph Attention Network (TGAN) with LSTM is incorporated to capture the sequential learning behavior of users over time. The system performs edge prediction to estimate the relevance between users and questions, generating personalized recommendations.

Beyond recommendation, the system also constructs a structured learning path by ordering recommended items from low to high confidence, helping learners progress in a meaningful way. An explainability module provides insights into model contributions and embedding similarities, improving transparency and trust.

The system is implemented as a fully functional web application using Streamlit, offering an interactive user interface with features such as recommendation display, learning path visualization, knowledge graph representation, and performance metrics.

Experimental results demonstrate strong performance in terms of Precision@K, Recall@K, NDCG, MAP, and AUC, indicating the effectiveness of the proposed approach. The project highlights the potential of graph-based deep learning methods in building adaptive, explainable, and scalable educational recommendation systems.

# CHAPTER – 1

# INTRODUCTION

## 1.1 Introduction of the Project

The rapid growth of digital learning platforms has significantly transformed the way students access, engage with, and consume educational content. In recent years, the proliferation of online learning environments, including Massive Open Online Courses (MOOCs), e-learning platforms, and interactive educational applications, has created unprecedented opportunities for learners across the globe. Students now have access to a vast array of resources such as practice questions, tutorial videos, concept explanations, quizzes, and interactive learning modules. This shift has democratized education by removing geographical and temporal barriers, allowing learners to study at their own pace and according to their individual preferences. However, while this abundance of content has enhanced accessibility and flexibility, it has also introduced a significant challenge: determining what to learn next.

In such content-rich environments, learners are often overwhelmed by the sheer volume of available materials. Without proper guidance or structured direction, they may struggle to identify which topics are most relevant to their current knowledge level and which learning path would be most effective for their progress. This lack of clarity can lead to confusion, inefficiency, and a fragmented learning experience. Learners may end up spending time on content that is either too basic or too advanced, thereby reducing the effectiveness of their learning efforts. Consequently, the need for intelligent systems that can guide learners through this vast landscape of educational content has become increasingly important.

Most existing online learning platforms attempt to address this issue through recommendation systems. These systems are designed to suggest relevant content to users based on their past interactions, preferences, or behaviors. Common approaches include popularity-based recommendations, which suggest widely accessed or highly rated content; recency-based recommendations, which prioritize recently added materials; and collaborative filtering techniques, which identify patterns among users with similar behaviors. While these methods provide a certain level of personalization, they are inherently limited in their ability to fully understand the complexity of the learning process.

One of the key limitations of traditional recommendation systems is that they rely on simple and often shallow representations of data. They treat user interactions as independent events and fail to capture the deeper relationships between learners, questions, and the underlying concepts or skills. For example, two questions may appear unrelated based on surface-level features but may actually test the same underlying concept. Similarly, two learners may have different interaction histories but share similar knowledge gaps. Traditional methods are not well-equipped to identify such hidden relationships, which can result in recommendations that are not aligned with a learner's actual needs.

Learning is inherently a structured and interconnected process. Concepts are not isolated entities; rather, they are linked through prerequisite relationships, hierarchical dependencies, and progression pathways. For instance, understanding advanced topics in subjects like mathematics or computer science often requires a strong foundation in basic principles. A learner cannot effectively grasp complex algorithms without first understanding fundamental

concepts such as data structures and logic. This interconnected nature of knowledge highlights the importance of modeling relationships between concepts in a meaningful way.

However, traditional recommendation systems typically treat data as flat and independent, ignoring these critical relationships. This limitation prevents them from providing structured guidance that aligns with the natural progression of learning. As a result, learners may receive recommendations that are either too advanced, lacking necessary prerequisites, or too simplistic, offering little value in terms of skill development. This gap underscores the need for more advanced approaches that can capture the structure and dependencies inherent in educational content.

To address these challenges, this project proposes an intelligent Learning Path Recommendation System using Graph Neural Networks (GNNs). The central idea behind this approach is to represent the learning environment as a graph, a data structure that naturally captures relationships between entities. In this graph, nodes represent different elements such as users (learners), questions (learning items), and skills (concepts), while edges represent interactions and relationships between these elements. For example, an edge may indicate that a user has attempted a particular question or that a question is associated with a specific skill.

This graph-based representation allows the system to model both direct and indirect relationships between entities. Unlike traditional methods, which consider only immediate interactions, graph-based approaches can capture multi-hop relationships, enabling a deeper understanding of the learning ecosystem. For instance, the system can identify connections between learners who have attempted similar questions, relationships between questions that share common skills, and dependencies between different concepts. This comprehensive representation provides a more realistic and holistic view of the learning process.

Graph Neural Networks are specifically designed to operate on graph-structured data. They extend traditional neural network architectures by incorporating the ability to aggregate information from neighboring nodes. This means that the representation of each node is influenced not only by its own features but also by the features of its neighbors and the overall structure of the graph. Through multiple layers of aggregation, GNNs can learn rich and informative embeddings that capture both local and global patterns within the graph.

By leveraging GNNs, the proposed system can identify complex patterns and relationships that are not easily captured by traditional methods. For example, it can detect similarities between learners based on shared learning behaviors, identify clusters of related questions, and uncover dependencies between different skills. This capability enables the system to generate more accurate, context-aware, and personalized recommendations.

In this project, multiple GNN architectures are employed to fully exploit the potential of graph-based learning. These include Graph Convolutional Networks (GCN), which perform convolution operations on graph data to aggregate information from neighboring nodes; Graph Attention Networks (GAT), which introduce attention mechanisms to assign different importance weights to neighbors; GraphSAGE, which enables inductive learning and scalability to large graphs; Graph Isomorphism Networks (GIN), which provide strong discriminative power for distinguishing graph structures; and Relational Graph Convolutional Networks (RGCN), which are capable of handling heterogeneous graphs with multiple types of relationships.

Each of these models contributes unique strengths to the system. By combining them using an ensemble approach, the system leverages their complementary capabilities, resulting in improved performance, robustness, and generalization. The ensemble method reduces the risk

of relying on a single model and ensures that the system can handle diverse learning scenarios effectively.

In addition to modeling static relationships, the project also incorporates a temporal component to capture the dynamic nature of learning behavior. Learning is not a static process; it evolves over time as learners interact with content, gain new knowledge, and refine their understanding. To model this temporal dimension, the system utilizes a Temporal Graph Attention Network (TGAN) combined with Long Short-Term Memory (LSTM) networks. The LSTM component captures sequential patterns in user interactions, while the attention mechanism highlights the most relevant past interactions. This integration enables the system to adapt recommendations based on the learner's current state, making them more timely and relevant.

The core task of the system is formulated as an edge prediction problem. In this context, the goal is to predict the likelihood that a user will interact with a particular question. This probabilistic formulation allows the system to assign scores to potential interactions, which can then be used to rank items and generate personalized recommendations. This approach provides a principled and data-driven framework for recommendation, ensuring that suggestions are based on learned patterns rather than heuristic rules.

Furthermore, the system enhances its practical usefulness by organizing recommended items into a structured learning path. Instead of presenting a random list of recommendations, the system arranges them in a logical sequence, typically progressing from lower confidence items to higher confidence ones. This ordering reflects the learner's current capabilities and provides a clear roadmap for progression. By guiding learners step-by-step through increasingly challenging content, the system supports more effective and efficient learning.

Another important aspect of this project is its focus on explainability. Many advanced machine learning models, particularly deep learning models, operate as black boxes, offering little insight into how decisions are made. In educational contexts, this lack of transparency can be a significant drawback, as learners and educators need to understand the reasoning behind recommendations. To address this issue, the proposed system incorporates explainability features such as model contribution analysis and embedding similarity measures. These features provide insights into why certain items are recommended, enhancing transparency and building user trust.

The system is implemented as a web-based application using Streamlit, which provides a simple yet powerful interface for user interaction. Users can input their ID, generate recommendations, visualize learning paths, explore knowledge graphs, and analyze performance metrics. The integration of backend machine learning models with an interactive frontend ensures a seamless and engaging user experience. This design makes the system accessible to users without requiring technical expertise.

Overall, this project aims to bridge the gap between traditional recommendation systems and the actual needs of modern learners. By combining graph-based learning, temporal modeling, ensemble techniques, and explainability, the proposed system offers a comprehensive and intelligent solution for personalized learning path recommendation. It demonstrates how advanced machine learning techniques can be effectively applied to enhance educational experiences and support better learning outcomes.

The proposed system not only improves recommendation accuracy but also aligns with the natural structure of learning, providing meaningful and actionable guidance. It addresses key challenges such as data sparsity, lack of structure, and absence of transparency, making it a valuable contribution to the field of educational technology.

In conclusion, the Learning Path Recommendation System based on Graph Neural Networks represents a significant step forward in the development of intelligent educational systems. By modeling the learning environment as a graph and leveraging advanced machine learning techniques, the system provides a powerful, adaptive, and user-centric approach to recommendation. It highlights the potential of graph-based methods in transforming how learners navigate educational content and paves the way for future innovations in personalized learning.

# CHAPTER – 2

# RELATED WORK

## 2.1 Related Work

The problem of recommending relevant educational content to learners has been extensively studied in the fields of machine learning, data mining, and educational data analysis. With the rapid growth of online learning platforms, there has been a significant increase in the availability of diverse educational resources, making personalization an essential requirement for effective learning. Researchers have proposed a wide range of recommendation techniques aimed at improving the learning experience by tailoring content to individual user needs and preferences.

In the early stages, recommendation systems primarily relied on simple heuristic-based methods such as popularity-based suggestions and rule-based filtering. These approaches were easy to implement but lacked the ability to provide personalized and context-aware recommendations. As the field progressed, more advanced techniques such as collaborative filtering and content-based recommendation methods were introduced. These methods improved personalization by analyzing user behavior and item characteristics, but they still operated on relatively simple data representations.

With advancements in deep learning, more sophisticated models have been developed to capture complex patterns in user interactions and learning data. These models have significantly improved recommendation accuracy and adaptability. However, despite these advancements, many existing approaches still face limitations, such as the inability to model complex relationships between users, learning materials, and underlying concepts.

These limitations highlight the need for more advanced solutions, such as graph-based recommendation systems, which can effectively capture the interconnected nature of educational data and provide more accurate, structured, and meaningful learning recommendations.

### Traditional Recommendation Systems

Early recommendation systems in educational platforms were primarily based on content-based filtering and collaborative filtering techniques. Content-based filtering recommends items that are similar to those previously interacted with by the user, using features such as topic, difficulty level, or type of question. This approach focuses on analyzing the characteristics of items and matching them with the user's past preferences. While content-based filtering is relatively simple to implement and offers a certain level of interpretability, it has notable limitations. One of its main drawbacks is the lack of diversity in recommendations, as it tends to repeatedly suggest similar types of content. As a result, learners may not be exposed

to new or unexplored concepts, which is essential for comprehensive learning and skill development.

Collaborative filtering, on the other hand, leverages interactions between users and items to identify patterns and similarities across a larger user base. It operates on the assumption that users with similar behaviors or preferences are likely to benefit from similar content. Techniques such as user-based similarity, item-based similarity, and matrix factorization have been widely adopted in this approach. These methods analyze historical interaction data to generate personalized recommendations, often achieving better performance than content-based methods in terms of accuracy and relevance.

However, collaborative filtering also suffers from several well-known challenges. One of the major issues is the cold-start problem, where the system struggles to provide accurate recommendations for new users or new items due to a lack of sufficient interaction data. Data sparsity is another significant concern, as most users interact with only a small subset of available content, making it difficult to identify meaningful patterns. Additionally, collaborative filtering methods are limited in their ability to capture complex relationships between concepts, as they primarily focus on interaction patterns rather than underlying knowledge structures.

In the context of education, these traditional approaches are particularly limited because they treat learning data as flat and independent. They fail to consider the hierarchical and interconnected nature of knowledge, where concepts are linked through prerequisites and dependencies. This limitation reduces their effectiveness in providing structured and meaningful learning guidance.

**Machine Learning-Based Approaches**

To overcome the limitations of traditional methods, researchers introduced machine learning techniques for recommendation systems. Algorithms such as decision trees, logistic regression, support vector machines, and neural networks have been widely applied to predict user preferences and learning outcomes. These approaches marked a significant improvement over earlier methods by enabling systems to learn patterns directly from data rather than relying solely on predefined rules.

In educational settings, machine learning models typically utilize features such as user performance, time spent on tasks, interaction history, and assessment results to generate recommendations. By analyzing these factors, the models can identify patterns in learning behavior and provide more personalized suggestions. Additionally, these techniques are capable of capturing nonlinear relationships within the data, leading to improved prediction accuracy compared to traditional approaches.

However, despite these advantages, machine learning models still have notable limitations. One key drawback is their reliance on handcrafted features, which require domain expertise and may not fully capture the complexity of relationships between users, questions, and concepts. These models do not inherently represent connections between entities, limiting their ability to model the structured nature of learning.

Another major limitation is the lack of interpretability. Many machine learning models function as black boxes, making it difficult to understand the reasoning behind specific recommendations. This lack of transparency reduces user trust and limits their effectiveness in real-world educational environments.

### Deep Learning-Based Recommendation Systems

With the advancement of deep learning, more sophisticated models such as Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM) networks, and Convolutional Neural Networks (CNNs) have been increasingly applied to recommendation tasks. These models have significantly improved the ability of systems to learn complex patterns from large-scale data, enabling more accurate and dynamic recommendations compared to traditional machine learning approaches.

In particular, sequential models like LSTM have gained prominence in educational data mining due to their ability to capture temporal dependencies in user interactions. Learning is inherently a time-dependent process, where a learner's knowledge evolves through continuous practice and engagement. LSTM networks are designed to handle such sequential data by maintaining memory of past interactions, allowing them to model how a learner's understanding changes over time. This makes them especially useful for predicting future performance and recommending appropriate learning materials based on recent activity.

However, despite their strengths in modeling sequences, these deep learning approaches have certain limitations. One key drawback is that they treat interactions as linear sequences, focusing primarily on temporal order while ignoring the structural relationships between different entities such as users, questions, and skills. As a result, they are unable to capture the interconnected nature of knowledge, where concepts are linked through dependencies and hierarchical relationships.

Additionally, deep learning models typically require large amounts of data for effective training, which may not always be available in educational settings. They also demand significant computational resources, including powerful hardware such as GPUs, making them less accessible for smaller institutions. Another important limitation is the lack of transparency, as these models often operate as black boxes, making it difficult to interpret and explain their predictions. This reduces user trust and limits their practical applicability in real-world learning environments.

### Knowledge Tracing and Educational Models

Another important area of research in educational recommendation is Knowledge Tracing (KT), which focuses on modeling a learner's knowledge state over time. Techniques such as Bayesian Knowledge Tracing (BKT) and Deep Knowledge Tracing (DKT) have been widely used to estimate how well a student has mastered specific skills based on their past interactions. These models analyze sequences of correct and incorrect responses to infer the probability of skill mastery, enabling predictions about future performance.

Bayesian Knowledge Tracing is a probabilistic approach that models learning as a hidden process, updating the learner's knowledge state as new observations are made. It is simple, interpretable, and effective for small-scale applications. Deep Knowledge Tracing, on the other hand, utilizes deep learning techniques, particularly recurrent neural networks, to capture complex patterns in student behavior. DKT can model nonlinear relationships and long-term dependencies, making it more powerful than traditional methods.

While knowledge tracing models are effective in predicting student performance, they are primarily focused on assessment rather than recommendation. Their main objective is to estimate whether a learner will correctly answer a question, rather than to suggest what

content should be learned next. As a result, they do not explicitly generate structured learning paths or provide comprehensive guidance for learners.

Additionally, these models do not consider relationships between different learners and questions at a global level. They typically operate on sequential data, focusing on individual learning trajectories without leveraging broader contextual information. Furthermore, knowledge tracing approaches do not utilize graph-based representations, limiting their ability to capture complex dependencies between concepts. This restricts their effectiveness in modeling the interconnected nature of knowledge in educational systems.

### Graph-Based Recommendation Systems

Recent advancements in recommendation systems have introduced graph-based approaches, which provide a more natural and effective way to model relationships between entities. Unlike traditional methods that treat data as flat and independent, graph-based systems represent users and items as nodes, while their interactions are modeled as edges. This representation forms a graph structure that captures the complex and interconnected nature of real-world data.

In such systems, nodes can represent various entities such as learners, questions, or skills, and edges can represent different types of relationships, including interactions, similarities, or dependencies. This flexible representation allows the system to incorporate multiple types of information within a unified framework. As a result, graph-based approaches can model both homogeneous and heterogeneous relationships, making them highly versatile for recommendation tasks.

One of the key advantages of graph-based methods is their ability to propagate information across nodes. Through this process, the system can aggregate information not only from direct neighbors but also from multi-hop connections within the graph. This enables the model to capture both direct and indirect relationships between entities. For example, it can identify similarities between users who have interacted with related content or uncover hidden connections between questions that share common underlying concepts.

These capabilities make graph-based approaches particularly suitable for educational applications, where knowledge is inherently interconnected and hierarchical. Concepts are often linked through prerequisite relationships, and understanding one topic may depend on mastering others. By effectively modeling these dependencies, graph-based recommendation systems can provide more accurate, context-aware, and structured learning recommendations, ultimately improving the overall learning experience.

### Graph Neural Networks (GNNs)

Graph Neural Networks (GNNs) have emerged as a powerful tool for learning from graph-structured data, offering significant advantages over traditional machine learning and deep learning approaches. These models are specifically designed to capture the relationships and dependencies between interconnected entities, making them highly suitable for recommendation systems. Several GNN architectures, such as Graph Convolutional Networks (GCN), Graph Attention Networks (GAT), GraphSAGE, and Graph Isomorphism Networks (GIN), have been successfully applied in this domain.

Graph Convolutional Networks (GCN) operate by aggregating information from neighboring nodes to learn meaningful node embeddings, allowing the model to capture local structural information. Graph Attention Networks (GAT) extend this idea by introducing attention mechanisms, which assign different importance weights to neighboring nodes, enabling the model to focus on the most relevant connections. GraphSAGE is designed for scalability and inductive learning, making it suitable for large graphs and dynamic environments. Graph Isomorphism Networks (GIN), on the other hand, provide strong representation power and are effective in distinguishing complex graph structures.

In educational recommendation systems, GNNs can model relationships between users, questions, and skills more effectively than traditional methods. By leveraging the structure of the learning graph, these models can capture both direct and indirect interactions, leading to richer and more informative representations. This results in improved recommendation accuracy and better alignment with the learner's needs.

However, most existing GNN-based systems focus primarily on static graphs and do not account for the temporal dynamics of learning behavior. This limitation restricts their ability to adapt to changes in a learner's knowledge over time.

**Temporal Graph Models**

Temporal graph models have been introduced to address the dynamic nature of user interactions in recommendation systems. Unlike static graph models, which assume that relationships remain constant over time, temporal graph models incorporate the element of time to capture how interactions and behaviors evolve. Prominent approaches in this area include Temporal Graph Networks (TGN) and Temporal Graph Attention Networks (TGAN), which extend traditional graph neural network architectures by integrating temporal information.

These models combine graph-based learning with sequence modeling techniques such as Long Short-Term Memory (LSTM) networks. By doing so, they are able to capture both the structural relationships between entities and the sequential patterns of interactions over time. This dual capability allows temporal graph models to learn how connections between users, items, and concepts change as new data becomes available.

Temporal models are particularly useful in educational settings, where a learner's knowledge state is not static but continuously evolves through practice, feedback, and exposure to new concepts. For example, a learner who initially struggles with a topic may gradually improve over time, and the recommendation system must adapt to reflect this change. By incorporating time-aware information, temporal graph models can better understand the progression of learning and adjust recommendations accordingly.

As a result, these models provide more adaptive, personalized, and context-aware recommendations. They ensure that suggested learning materials align with the learner's current level of understanding, making them highly effective for supporting dynamic and continuous learning processes.

**Limitations of Existing Work**

Despite significant progress, existing approaches still have several limitations:

- Many systems rely on single models and lack robustness
- Limited integration of structural (graph) and temporal information
- Lack of explainability in recommendations

- Absence of structured learning path generation
- Difficulty in scaling to large educational datasets

Most systems focus only on predicting user-item interactions without organizing recommendations into meaningful learning sequences.

**Motivation for the Proposed System**

The limitations of existing systems highlight the need for a more comprehensive solution that can:

- Capture complex relationships using graph structures
- Incorporate temporal dynamics of learning
- Combine multiple models for better performance
- Provide explainable recommendations
- Generate structured and personalized learning paths

This project addresses the limitations of existing recommendation systems by integrating multiple Graph Neural Network (GNN) models, temporal learning mechanisms, and explainability features into a unified and cohesive framework. By combining these advanced techniques, the proposed system aims to deliver a more accurate, adaptive, and transparent recommendation solution tailored for modern educational platforms. The use of multiple GNN architectures enables the system to capture diverse aspects of graph-structured data, while temporal learning ensures that recommendations evolve in accordance with the learner's changing knowledge state. Additionally, the inclusion of explainability components allows users to understand the reasoning behind recommendations, thereby increasing trust and usability. The unified framework is designed to overcome the shortcomings of traditional and existing methods, such as the inability to model complex relationships, lack of adaptability to dynamic learning behavior, and absence of structured guidance. By addressing these challenges, the system provides not only personalized recommendations but also meaningful learning paths that guide users through content in a logical and effective manner.

Overall, the evolution of recommendation systems from traditional heuristic-based approaches to advanced graph-based deep learning models reflects the growing demand for intelligent and adaptive learning systems. As educational data becomes more complex and interconnected, there is a need for methods that can effectively capture these characteristics. The proposed work builds upon recent advancements in graph neural networks and extends them by incorporating temporal modeling and explainability. This integration results in a next-generation learning path recommendation system that is more robust, scalable, and aligned with the real-world needs of learners and educational platforms.

# CHAPTER – 3

# PROBLEM STATEMENT

## 3.1 Problem Statement

With the rapid expansion of online learning platforms, students today are exposed to an enormous and ever-growing volume of educational content, including practice questions, tutorial videos, interactive exercises, and concept-based learning materials. This abundance of resources has significantly transformed the way learners access knowledge, offering greater flexibility, accessibility, and personalization than traditional classroom environments. However, despite these advantages, it also introduces a critical challenge: learners often struggle to identify which content is most relevant to their current knowledge level and, more importantly, what they should learn next. Without proper guidance, students may feel overwhelmed by the sheer amount of available information, leading to inefficient learning, confusion, lack of direction, and ultimately reduced engagement and motivation.

To address this issue, many existing educational platforms have implemented recommendation systems designed to guide learners toward suitable content. While these systems provide some level of assistance, the majority of them rely on traditional approaches such as popularitybased recommendations, content similarity methods, or collaborative filtering techniques. Popularitybased systems suggest widely used or highly rated content, assuming it will be beneficial for all learners. Content-based methods recommend items similar to those previously interacted with, while collaborative filtering identifies patterns among users with similar behaviors. Although these methods are relatively simple and effective in certain contexts, they operate on flat data representations and treat user interactions as independent and isolated events. As a result, they fail to capture the deeper and more complex relationships that exist between learners, questions, and underlying skills.

One of the most significant limitations of current recommendation systems is their inability to model the interconnected and hierarchical nature of learning. In real-world educational scenarios, knowledge is not acquired in isolation but is structured as a network of dependencies, where mastering one concept often requires understanding several prerequisite topics. For example, solving advanced problems in machine learning requires a solid foundation in mathematics and statistics. Traditional recommendation techniques do not consider these dependencies, which can lead to inappropriate recommendations that are either too advanced, too basic, or not aligned with the learner's progression. This lack of contextual awareness reduces the effectiveness of the learning process and may hinder the development of a strong conceptual foundation.

Another major challenge faced by existing systems is the issue of data sparsity and the cold-start problem. In many cases, learners interact with only a small subset of available content, resulting in limited data for generating accurate recommendations. This sparsity makes it difficult for traditional models to identify meaningful patterns and relationships. The problem becomes even more severe for new users, who have little to no interaction history. Without sufficient data, recommendation systems struggle to provide personalized suggestions, leading to poor initial user experiences and decreased engagement. Addressing this issue is crucial for ensuring that the system remains effective across a diverse range of users.

In addition to these challenges, most existing recommendation systems do not adequately incorporate temporal learning behavior. A learner's knowledge state is dynamic and evolves over time as they practice, revise, and acquire new skills. However, many traditional models treat all interactions equally, regardless of when they occurred. This static approach fails to reflect the learner's current level of understanding and may result in outdated or irrelevant

recommendations. For instance, a concept that was difficult for a learner in the past may no longer require attention after sufficient practice. Ignoring such temporal dynamics reduces the adaptability and responsiveness of the recommendation system.

Another critical limitation is the lack of explainability in many modern recommendation systems. Advanced machine learning and deep learning models often function as black boxes, providing recommendations without offering clear insights into the reasoning behind them. In the context of education, explainability is particularly important, as both learners and educators need to understand why certain content is being suggested. Transparent recommendations help build trust in the system, enable users to make informed decisions, and allow educators to validate and improve the model's performance. Without explainability, users may question the relevance of recommendations and become less likely to rely on the system.

Furthermore, existing systems typically generate a list of recommended items without organizing them into a meaningful and structured sequence. Learning is inherently a step-by-step process that requires a logical progression from basic to advanced concepts. Simply presenting a list of recommended questions or topics does not provide learners with a clear roadmap for achieving their goals. The absence of a well-defined learning path limits the practical usefulness of recommendations and reduces their impact on actual learning outcomes. A structured approach is essential for guiding learners effectively and ensuring steady progress.

Another important limitation is the lack of integration between multiple advanced modeling techniques. Many current systems rely on a single algorithm or model, which may not be sufficient to capture all aspects of complex educational data. For example, one model may excel at capturing user-item interactions, while another may be better suited for modeling temporal behavior or concept dependencies. Relying on a single approach reduces the robustness of the system and makes it more susceptible to noise, incomplete data, and varying user behaviors. A more comprehensive solution should combine multiple techniques to leverage their respective strengths.

Considering all these challenges, there is a clear need for an advanced recommendation system that goes beyond traditional approaches. Such a system should be capable of modeling complex relationships between users, questions, and skills, effectively handling sparse and dynamic data, providing transparent and explainable recommendations, and generating structured learning paths that align with the learner's progression. Addressing these requirements is essential for improving the overall learning experience and ensuring that learners receive meaningful and personalized guidance.

Therefore, the problem addressed in this project is to design and develop an intelligent learning path recommendation system that leverages graph-based modeling to capture the intricate relationships between learners and educational content. The system should incorporate temporal dynamics to reflect evolving user behavior, utilize multiple models to enhance accuracy and robustness, and provide explainable and structured recommendations that support effective learning.

To achieve this, the proposed solution utilizes Graph Neural Networks (GNNs) to model the learning environment as a graph, where nodes represent users, questions, and concepts, and edges represent their interactions and dependencies. This graph-based approach enables the system to capture complex relationships that traditional methods cannot effectively represent. By integrating multiple GNN models along with temporal learning techniques and explainability features, the system aims to deliver accurate, adaptive, and user-centric recommendations.

Overall, this project addresses a critical gap in existing learning systems by transforming unstructured and static recommendation approaches into a structured, intelligent, and explainable framework. By aligning with the real-world needs of learners and incorporating advanced modeling techniques, the proposed system has the potential to significantly enhance the effectiveness, engagement, and personalization of modern educational platforms.

# CHAPTER – 4

# SYSTEM ANALYSIS

## 4.1 Requirement Analysis

Requirement analysis is a crucial phase in the development of any system, as it defines the functional and non-functional needs that the system must satisfy. In the context of the proposed Learning Path Recommendation System, requirement analysis helps in identifying the essential features, system constraints, and performance expectations required to build an efficient and user-friendly solution.

The requirements of the system are categorized into two main types: **Functional Requirements** and **Non-Functional Requirements**.

### Functional Requirements

Functional requirements describe the core functionalities that the system must perform. These define what the system should do in order to achieve its objectives.

- The system must allow users to input a valid **user ID** to generate personalized recommendations.
- The system should process user interaction data and map it into a graph structure consisting of users, questions, and skills.
- The system must generate **personalized recommendations** based on the learned graph embeddings.
- The system should provide a **top-K list of recommended questions** relevant to the user.
- The system must generate a **structured learning path**, ordering recommended items from lower confidence to higher confidence to guide the user step-by-step.
- The system should support multiple Graph Neural Network models such as GCN, GAT, GraphSAGE, GIN, and RGCN for learning representations.
- The system must integrate a **temporal model (TGAN with LSTM)** to capture user learning behavior over time.
- The system should perform **edge prediction** to estimate the probability of interaction between users and questions.
- The system must combine outputs from multiple models using an **ensemble approach** to improve accuracy and robustness.
- The system should display **prediction scores** for each recommended item.
- The system must provide **explainability features**, including similarity scores and model contribution insights.

- The system should visualize the relationships using a **knowledge graph representation**.
- The system must provide an interactive **user interface (UI)** for easy access to recommendations and visualizations.
- The system should allow users to select the number of recommendations (Top-K).
- The system must load pre-trained models and perform inference efficiently.

**Non-Functional Requirements**

Non-functional requirements define how the system performs and the quality attributes it must satisfy.

- **Performance:**
The system should generate recommendations in real-time or near real-time (preferably within a few seconds).
- **Scalability:**
The system should handle a large number of users, questions, and interactions without significant performance degradation.
- **Accuracy:**
The recommendation system should achieve high accuracy and strong ranking performance (Precision@K, Recall@K, NDCG, MAP, AUC).
- **Usability:**
The system should provide a simple, intuitive, and interactive interface that can be easily used by both technical and non-technical users.
- **Reliability:**
The system should consistently produce stable and correct outputs without failure.
- **Maintainability:**
The system should be modular in design, allowing easy updates, debugging, and integration of new models or features.
- **Flexibility:**
The system should support the addition of new datasets, models, and features without major redesign.
- **Portability:**
The system should run across different environments such as local machines, cloud platforms, and notebook environments like Kaggle or Colab.
- **Security:**
The system should ensure safe handling of user data and prevent unauthorized access to the system.
- **Interpretability:**
The system should provide clear explanations for recommendations to improve user trust and transparency.

The requirement analysis highlights the need for a system that is not only accurate but also scalable, interpretable, and user-friendly. By clearly defining both functional and non-functional

requirements, the foundation is established for designing and implementing a robust learning path recommendation system using advanced graph-based techniques.

### 4.2 Risk Analysis

Risk analysis is an essential part of system development, as it helps identify potential challenges that may arise during the design, implementation, and deployment phases. By analyzing these risks in advance, appropriate mitigation strategies can be planned to ensure the system operates reliably and efficiently.

The proposed Learning Path Recommendation System involves multiple components such as data processing, graph construction, model training, and user interface integration. Each of these components introduces certain risks that must be carefully managed.

**Identified Risks and Mitigation Strategies**

**1. Data Sparsity Risk**

- Description:

Users may have limited interaction data, leading to insufficient information for accurate recommendations.

- Impact: High
- Probability: Medium

- Mitigation Strategy:

Use graph-based modeling to propagate information across connected nodes. Incorporate multiple users and relationships to reduce sparsity effects. Use embedding techniques to generalize patterns.

**2. Model Overfitting**

- Description:

The model may perform well on training data but fail to generalize to unseen data.

- Impact: High
- Probability: Medium

- Mitigation Strategy:

Apply regularization techniques, dropout layers, and proper validation strategies. Use train-test splits and monitor performance metrics to avoid overfitting.

**3. High Computational Complexity**

- Description:

Training multiple GNN models and handling large graphs may require high computational resources.

- Impact: Medium
- Probability: Medium

- Mitigation Strategy:

Limit dataset size during training, use batch processing, and optimize model parameters. Utilize GPU acceleration where available.

**4. Model Integration Complexity**

- Description:

Combining multiple GNN models and a temporal model into an ensemble may lead to integration challenges.

- Impact: Medium
- Probability: Medium

- Mitigation Strategy:

Adopt a modular architecture where each model is developed and tested independently before integration. Use consistent input-output formats.

**5. Inaccurate Recommendations**

- Description:

The system may generate irrelevant or less useful recommendations due to imperfect learning.

- Impact: High
- Probability: Medium

- Mitigation Strategy:

Use ensemble learning to improve prediction quality. Evaluate using multiple metrics such as Precision@K and NDCG. Continuously refine model parameters.

**6. Lack of Explainability**

- Description:

Users may not trust recommendations if the reasoning behind them is unclear.

- Impact: Medium
- Probability: Low

- Mitigation Strategy:

Provide similarity scores and model contribution insights. Include visual explanations and graph-based reasoning.

**7. UI Usability Issues**

- Description:

A complex or unclear interface may reduce user engagement and usability.

- Impact: Medium
- Probability: Medium

- Mitigation Strategy:

Design a simple and intuitive Streamlit interface. Conduct testing and improve based on feedback.

**8. System Performance Delay**

• Description:
Slow response time during recommendation generation may affect user experience.

• Impact: Medium
• Probability: Low

• Mitigation Strategy:
Use pre-trained models and efficient inference pipelines. Optimize data loading and processing.

**9. Dependency on Dataset Quality**

• Description:
Poor quality or biased data may affect model performance.

• Impact: High
• Probability: Medium

• Mitigation Strategy:
Perform proper data preprocessing, cleaning, and validation. Use representative datasets.

The risk analysis highlights potential challenges related to data, model performance, system complexity, and user interaction. By identifying these risks early and implementing appropriate mitigation strategies, the system can be made more robust, reliable, and effective. Proper handling of these risks ensures smooth development and successful deployment of the learning path recommendation system.

### 4.3 Feasibility Analysis

Feasibility analysis is conducted to evaluate whether the proposed system can be successfully developed and deployed within given constraints. It plays a crucial role in determining the practicality of a project before full-scale implementation. This analysis examines multiple dimensions, including technical capability, operational practicality, economic viability, and time constraints, to ensure that the system is realistic, sustainable, and efficient. The proposed Learning Path Recommendation System using Graph Neural Networks (GNNs) has been carefully analyzed across these dimensions to validate its feasibility and effectiveness in real-world educational environments.

**Technical Feasibility**

The proposed system is technically feasible as it is built using widely available, reliable, and well-supported technologies. The core implementation is based on Python, which is one of the most widely used programming languages in machine learning and data science. The system leverages powerful frameworks such as PyTorch and PyTorch Geometric, which provide robust support for building and training Graph Neural Networks. These frameworks are actively maintained, well-documented, and supported by a large developer community, ensuring ease of development and troubleshooting.

In addition to GNN frameworks, the system utilizes essential data processing and analysis libraries such as NumPy, Pandas, and Scikit-learn. These libraries enable efficient handling of large datasets, preprocessing of user interactions, and evaluation of model performance using standard metrics. Their integration ensures that the data pipeline is both efficient and scalable.

The system incorporates multiple advanced GNN models, including Graph Convolutional Networks (GCN), Graph Attention Networks (GAT), GraphSAGE, Graph Isomorphism Networks (GIN), and Relational Graph Convolutional Networks (RGCN). Each of these models is designed to capture different aspects of graph-structured data, such as node relationships, attention mechanisms, and heterogeneous interactions. The availability of pre-implemented versions of these models in PyTorch Geometric significantly reduces development complexity and allows for efficient experimentation and optimization.

Furthermore, the inclusion of a temporal component using Long Short-Term Memory (LSTM) networks enhances the system's ability to model sequential learning behavior. LSTM models are well-established in handling time-dependent data and can be seamlessly integrated with GNN architectures. This combination allows the system to capture both structural and temporal patterns in learner interactions, improving recommendation accuracy.

The frontend of the system is developed using Streamlit, a lightweight and user-friendly framework for building interactive web applications. Streamlit enables rapid prototyping and deployment without requiring expertise in traditional web development technologies such as HTML, CSS, or JavaScript. This simplifies the development process while still providing an intuitive interface for users.

The backend inference pipeline is designed to be lightweight and efficient. Once the models are trained, generating recommendations requires minimal computational resources, making the system suitable for real-time or near real-time usage. The system can be executed on various platforms, including local machines, Google Colab, and Kaggle, which enhances its accessibility and flexibility.

Additionally, the availability of GPU acceleration further strengthens technical feasibility by significantly reducing training time for complex models. Even in the absence of GPUs, the system can still function effectively on CPUs, albeit with longer training durations. This flexibility ensures that the system can be implemented in diverse environments with varying computational resources.

Overall, from a technical perspective, the system is highly feasible, as it relies on mature technologies, scalable frameworks, and well-established methodologies that collectively support efficient development and deployment.

**Operational Feasibility**

The proposed system is designed with a strong focus on usability and practicality, making it operationally feasible for real-world applications. One of the key strengths of the system is its user-friendly interface, developed using Streamlit. This interface allows users to interact with the system effortlessly by entering their user ID and generating personalized recommendations without requiring any technical expertise.

The system provides clear and meaningful outputs, including recommended questions, structured learning paths, prediction scores, and explanations for recommendations. These outputs are presented in an intuitive format, enabling learners to easily understand and follow the suggested learning path. Educators can also benefit from these insights by monitoring student progress and identifying areas where additional support may be needed.

An important feature of the system is the visualization of the knowledge graph, which illustrates the relationships between users, questions, and concepts. This visualization enhances transparency and helps users understand how recommendations are generated. It also provides valuable insights into the structure of the learning domain, making the system more engaging and informative.

The system follows a modular architecture, where different components such as data preprocessing, graph construction, model training, inference, and visualization operate independently. This modular design improves maintainability and allows for easy updates and extensions. For example, new models can be added or existing components can be modified without affecting the entire system.

Another aspect contributing to operational feasibility is the system's minimal infrastructure requirements. It does not depend on complex distributed systems or specialized hardware, making it suitable for deployment in educational institutions with limited technical resources. The system can be integrated into existing learning platforms or used as a standalone application.

User adoption is also facilitated by the simplicity of the interface and the relevance of the recommendations. Since the system directly addresses the problem of identifying what to learn next, it provides immediate value to users, increasing the likelihood of acceptance and continued usage.

In summary, the system is operationally feasible due to its intuitive design, modular structure, minimal infrastructure requirements, and practical usefulness in educational settings.

**Economic Feasibility**

The proposed system is highly cost-effective, making it economically feasible for a wide range of users, including educational institutions, online learning platforms, and individual learners. One of the primary reasons for its affordability is the use of open-source technologies throughout the development process. Libraries such as PyTorch, PyTorch Geometric, Scikit-learn, Pandas, and Streamlit are freely available and do not require any licensing fees.

The hardware requirements for the system are relatively modest. While access to GPUs can accelerate model training, it is not a strict requirement for deployment. The system can run on standard personal computers or cloud-based platforms that offer free or low-cost computational resources, such as Google Colab and Kaggle. This significantly reduces the need for investment in expensive infrastructure.

Development costs are also minimized due to the availability of pre-built libraries and frameworks. These tools reduce the need for building components from scratch, allowing developers to focus on system integration and optimization. Additionally, the modular architecture of the system simplifies maintenance and future upgrades, further reducing longterm costs.

The system's scalability ensures that it can handle increasing amounts of data and users without requiring significant additional investment. This makes it suitable for both small-scale and large-scale deployments. Educational institutions can adopt the system without incurring substantial financial burdens, making it an attractive solution for improving learning outcomes.

Moreover, the potential benefits of the system, such as improved learning efficiency, better student engagement, and personalized guidance, can lead to indirect cost savings by reducing dropout rates and enhancing overall educational effectiveness.

Overall, the system is economically feasible due to its reliance on open-source tools, minimal hardware requirements, low development and maintenance costs, and high potential return on investment.

**Schedule Feasibility**

The development of the proposed system can be completed within a reasonable and

manageable timeframe. The project is structured into well-defined phases, including data preprocessing, graph construction, model development, training, evaluation, and frontend implementation. This structured approach ensures that each stage is clearly defined and can be executed systematically.

One of the key advantages of the project is that many components can be developed in parallel. For instance, while one part of the team focuses on data preprocessing and graph construction, another can work on model implementation and training. Similarly, frontend development using Streamlit can proceed alongside backend model development. This parallel workflow significantly reduces overall development time.

The use of established libraries and frameworks further accelerates the development process. Instead of implementing complex algorithms from scratch, developers can utilize existing implementations and customize them as needed. This not only saves time but also reduces the risk of errors.

Integration of different components is also straightforward, as both the frontend and backend are implemented in Python-based environments. This compatibility simplifies communication between components and reduces integration challenges. Testing and validation can be performed incrementally at each stage, ensuring that issues are identified and resolved early in the development cycle.

The availability of datasets such as EdNet further supports schedule feasibility by providing ready-to-use data for training and evaluation. This eliminates the need for extensive data collection, which can be time-consuming.

Additionally, the modular design allows for iterative development, where the system can be gradually improved and extended over time. Even if certain advanced features are not fully implemented in the initial phase, they can be added in future iterations without disrupting the existing system.

Given the structured development plan, availability of tools and resources, and potential for parallel execution, the project can be completed within the planned schedule without significant delays.

**Conclusion of Feasibility Analysis**

The feasibility analysis confirms that the proposed Learning Path Recommendation System is viable across all major dimensions, including technical, operational, economic, and scheduling aspects. The use of modern machine learning techniques, particularly Graph Neural Networks combined with temporal modeling, ensures that the system is both innovative and effective.

The reliance on accessible and well-supported tools makes the system practical to implement, while the user-friendly design ensures ease of adoption in real-world educational environments. The low cost of development and deployment further enhances its suitability for a wide range of users.

By addressing key challenges in existing recommendation systems and providing a structured, explainable, and adaptive solution, the proposed system establishes a strong foundation for successful implementation. This feasibility analysis demonstrates that the project is not only theoretically sound but also practically achievable, paving the way for its design, development, and deployment.

# CHAPTER – 5

# PROPOSED SOLUTION

## 5.1 Proposed Solution / Approach / Technique

The proposed system aims to address the limitations of traditional recommendation systems by introducing a graph-based, multi-model learning framework for generating personalized and structured learning paths. The system leverages Graph Neural Networks (GNNs), temporal modeling, and ensemble learning to capture complex relationships and evolving user behavior in educational data.

### 1. Graph-Based Representation of Learning

At the core of the proposed solution is the transformation of learning data into a graph structure. Unlike traditional systems that treat data as flat tables, this approach models the learning environment as an interconnected network.

- Nodes:
  - Users (learners)
  - Questions (learning items)
  - Skills (concepts/topics)
- Edges:
  - User → Question (interaction)
  - Question → Skill (concept mapping)

This representation allows the system to capture dependencies between concepts, relationships between users, and connections across different learning elements. It provides a more realistic modeling of the learning process.

### 2. Graph Neural Network Models

To learn from the graph structure, multiple Graph Neural Network models are employed:

- **GCN (Graph Convolutional Network):**
  Captures overall structural information by aggregating features from neighboring nodes.

- **GAT (Graph Attention Network):**
  Assigns different importance to neighbors using attention mechanisms.

- **GraphSAGE:**
  Efficiently samples and aggregates neighbor information, making it scalable for large graphs.

- **GIN (Graph Isomorphism Network):**
  Provides strong representational power to distinguish complex graph structures.

- **RGCN (Relational GCN):**
  Handles multiple types of relationships between nodes effectively.

Each model contributes unique strengths in learning graph representations, enabling the system to capture diverse patterns in the data.

### 3. Temporal Learning with TGAN

Learning is a dynamic process, where user knowledge evolves over time. To capture this temporal behavior, the system incorporates a **Temporal Graph Attention Network (TGAN)** combined with an **LSTM (Long Short-Term Memory)** model.

This component processes sequential interaction data and learns patterns in user behavior over time. It helps the system understand how a learner's performance changes and adapts recommendations accordingly.

### 4. Edge Prediction Mechanism

The recommendation task is formulated as an **edge prediction problem**, where the goal is to estimate the likelihood of interaction between a user and a question.

Process:

- Each node is represented as a vector (embedding)
- User and question embeddings are combined
- The combined vector is passed through a neural network (MLP)
- The output is a probability score

This score indicates how relevant a question is for a particular user. Higher scores correspond to better recommendations.

### 5. Ensemble Learning Approach

Instead of relying on a single model, the system uses an ensemble of multiple GNN models and the temporal model.

- Each model generates prediction scores independently
- The scores are combined using averaging

Final Score = (Sum of all model predictions) / Number of models

This approach improves:

- Accuracy
- Robustness
- Generalization

By combining multiple perspectives, the system reduces the limitations of individual models.

### 6. Learning Path Generation

Beyond generating recommendations, the system organizes them into a structured learning path.

- Recommended items are sorted based on prediction scores
- Lower scores → More challenging or less certain
- Higher scores → Easier or more suitable

The final learning path follows a progression:

Low Confidence → High Confidence

This helps learners move step-by-step and improves learning efficiency.

### 7. Explainability Module

To improve transparency, the system provides explainability features.

- Displays prediction scores
- Computes similarity between user and item embeddings
- Shows model contributions to final predictions

This helps users understand why certain recommendations are generated and builds trust in the system.

**8. System Implementation**

The system is implemented as a complete web-based application:

- **Backend:**

Handles data processing, model inference, and recommendation generation using PyTorch and PyTorch Geometric.

- **Frontend (Streamlit):**

Provides an interactive interface where users can:

- Input user ID
- View recommendations
- Explore learning paths
- Visualize knowledge graphs
- Analyze model outputs

The integration of backend models with a user-friendly frontend ensures practical usability.

The proposed solution combines graph-based modeling, multiple GNN architectures, temporal learning, and ensemble techniques to create a powerful recommendation system. By addressing the limitations of traditional approaches, the system provides accurate, adaptive, and explainable learning recommendations. It transforms raw interaction data into meaningful learning paths, making it highly suitable for modern educational platforms.

# CHAPTER – 6

# SYSTEM DESIGN

## 6.1 System Architecture

The proposed Learning Path Recommendation System is designed using a modular and layered architecture that ensures scalability, flexibility, and efficient processing of graph-based data. The architecture integrates data processing, graph modeling, machine learning inference, and user interaction into a unified system. It is broadly divided into three main layers: the **Frontend Layer**, the **Backend Layer**, and the **Model & Data Layer**.

**1. Frontend Layer (User Interface)**

The frontend layer is responsible for user interaction and visualization of results. It is implemented using **Streamlit**, which provides a simple and interactive web-based interface.

**Role:**

The primary role of the frontend is to allow users to interact with the system, input data, and view recommendations in an intuitive manner.

**Key Features:**

• User Input Interface:
Users can enter their **user ID** and select the number of recommendations (Top-K).

• Recommendation Display:
The system displays a list of recommended questions along with their prediction scores.

• Learning Path Visualization:
Recommended items are presented in a structured sequence to guide the learner step-by-step.

• Knowledge Graph Visualization:
The frontend provides a graphical representation of relationships between users, questions, and skills.

• Explainability Panel:
Displays similarity scores and insights into model predictions.

• Performance Metrics:
Shows evaluation metrics such as Precision, Recall, and NDCG for analysis.

This layer ensures that complex backend computations are presented in a clear and user-friendly manner.

**2. Backend Layer (Application Logic)**

The backend layer handles the core processing logic of the system. It acts as the bridge between the frontend and the machine learning models.

**Role:**
The backend is responsible for data preprocessing, model loading, inference, and generation of recommendations.

**Key Functions:**

• Data Preprocessing:
Converts raw interaction data into a structured format suitable for graph construction.

• Graph Construction:
Builds a graph with nodes representing users, questions, and skills, and edges representing interactions.

• Model Loading:
Loads pre-trained models and embeddings from saved files (e.g., .pth).

• Inference Engine:
Runs multiple GNN models and the temporal model to generate embeddings and prediction scores.

• Ensemble Processing:
Combines outputs from different models to produce a final recommendation score.

• Recommendation Generation:
Ranks items based on predicted scores and selects top-K recommendations.

• Learning Path Generation:
Organizes recommendations into a structured sequence.

The backend ensures efficient execution of all computational tasks and provides results to the frontend.

**3. Model & Data Layer**

This layer contains the core machine learning models and datasets used in the system. It is the computational backbone of the architecture.

**Components:**

• Dataset:
Educational interaction data containing user IDs, question IDs, timestamps, and correctness information.

• Graph Representation:
A graph structure consisting of:

- User nodes
- Question nodes
- Skill nodes

• Node Embeddings:
Each node is represented as a vector learned during training.

• GNN Models:
Multiple models including:

- GCN
- GAT
- GraphSAGE
- GIN
- RGCN

• Temporal Model:
TGAN with LSTM to capture sequential learning behavior.

• Edge Prediction Module:
A neural network that predicts the likelihood of interaction between users and questions.

• Saved Model Files:
Pre-trained weights and mappings stored in .pth format for efficient inference.

This layer is responsible for learning meaningful representations and generating predictions.

**System Workflow**

The overall workflow of the system can be summarized as follows:

1. The user inputs a user ID through the frontend interface.
2. The backend retrieves the corresponding user node from the graph.
3. Node embeddings are passed through multiple GNN models and the temporal model.
4. Each model generates prediction scores for user-question pairs.

5. Scores from all models are combined using an ensemble approach.
6. Top-K questions are selected based on final scores.
7. A learning path is generated by ordering the recommendations.
8. Results are displayed on the frontend along with visualizations and explanations.

**Architectural Advantages**

The proposed architecture offers several advantages:

- **Modularity:**
Each component can be developed and updated independently.

- **Scalability:**
The system can handle large datasets and complex graphs.

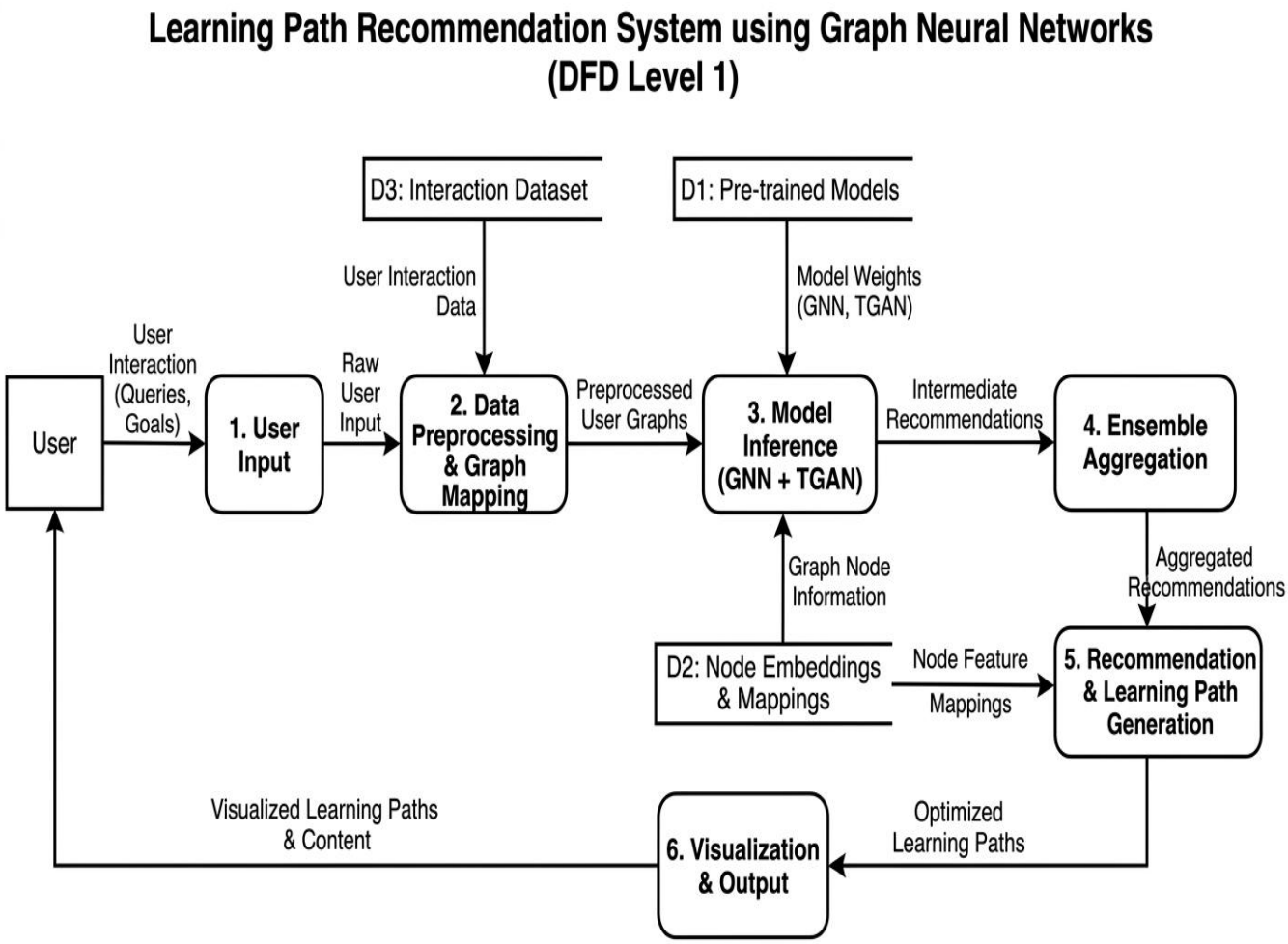
- **Flexibility:**
New models or features can be easily integrated.

- **Efficiency:**
Pre-trained models enable fast inference.

- **User-Friendliness:**
Interactive frontend simplifies usage.

The system architecture effectively integrates graph-based learning, temporal modeling, and user interaction into a cohesive framework. By separating responsibilities across layers, the design ensures maintainability, scalability, and efficient performance. This architecture forms the foundation for building an intelligent and practical learning path recommendation system.

### 6.2 Data Flow Diagram (DFD)




The Data Flow Diagram (DFD) represents how data moves through the Learning Path Recommendation System, illustrating the interaction between users, backend processing, machine learning models, and output visualization. It provides a clear understanding of how input data is transformed into meaningful recommendations.

The system follows a structured pipeline consisting of multiple processes that convert raw user input into personalized learning paths.

**Overview of Data Flow**

The data flow begins with the user providing input through the frontend interface and ends with the display of recommendations and visual insights. The system processes data through several stages, including preprocessing, model inference, aggregation, and visualization.

**Key Processes in the DFD**

**1. User Input (Process 1.1)**

The process starts when the user enters their **user ID** and selects the number of recommendations (Top-K) through the Streamlit interface. This input acts as the trigger for the recommendation pipeline.

**Output:** User input data sent to backend

**2. Data Preprocessing & Graph Mapping (Process 1.2)**

The backend processes the input by mapping the user ID to the corresponding node in the graph. It prepares the data by retrieving node embeddings and required mappings.

**Operations include:**
- Mapping user ID to graph node
- Retrieving embeddings
- Preparing input tensors

**Output:** Processed feature data

**3. Model Inference (Process 1.3)**

The processed data is passed to multiple models:

- GNN Models (GCN, GAT, SAGE, GIN, RGCN)
- Temporal Model (TGAN with LSTM)

Each model independently generates prediction scores for user-question pairs.

**Output:** Individual model predictions

**4. Ensemble Aggregation (Process 1.4)**

The outputs from all models are combined using an ensemble approach.

**Steps:**
- Collect scores from all models
- Compute average score
- Generate final prediction

This improves accuracy and robustness.

**Output:** Final prediction scores

**5. Recommendation & Learning Path Generation (Process 1.5)**

The system ranks items based on final scores and selects top-K recommendations.

Additionally, a structured learning path is generated by ordering items from lower confidence to higher confidence.

**Output:** Ranked recommendations + learning path

**6. Visualization & Output (Process 1.6)**

The final results are displayed on the frontend.

**Includes:**
- Recommended questions
- Scores
- Learning path
- Knowledge graph
- Explainability insights

This allows users to understand both results and reasoning.

**Data Stores**

The system uses the following data stores:

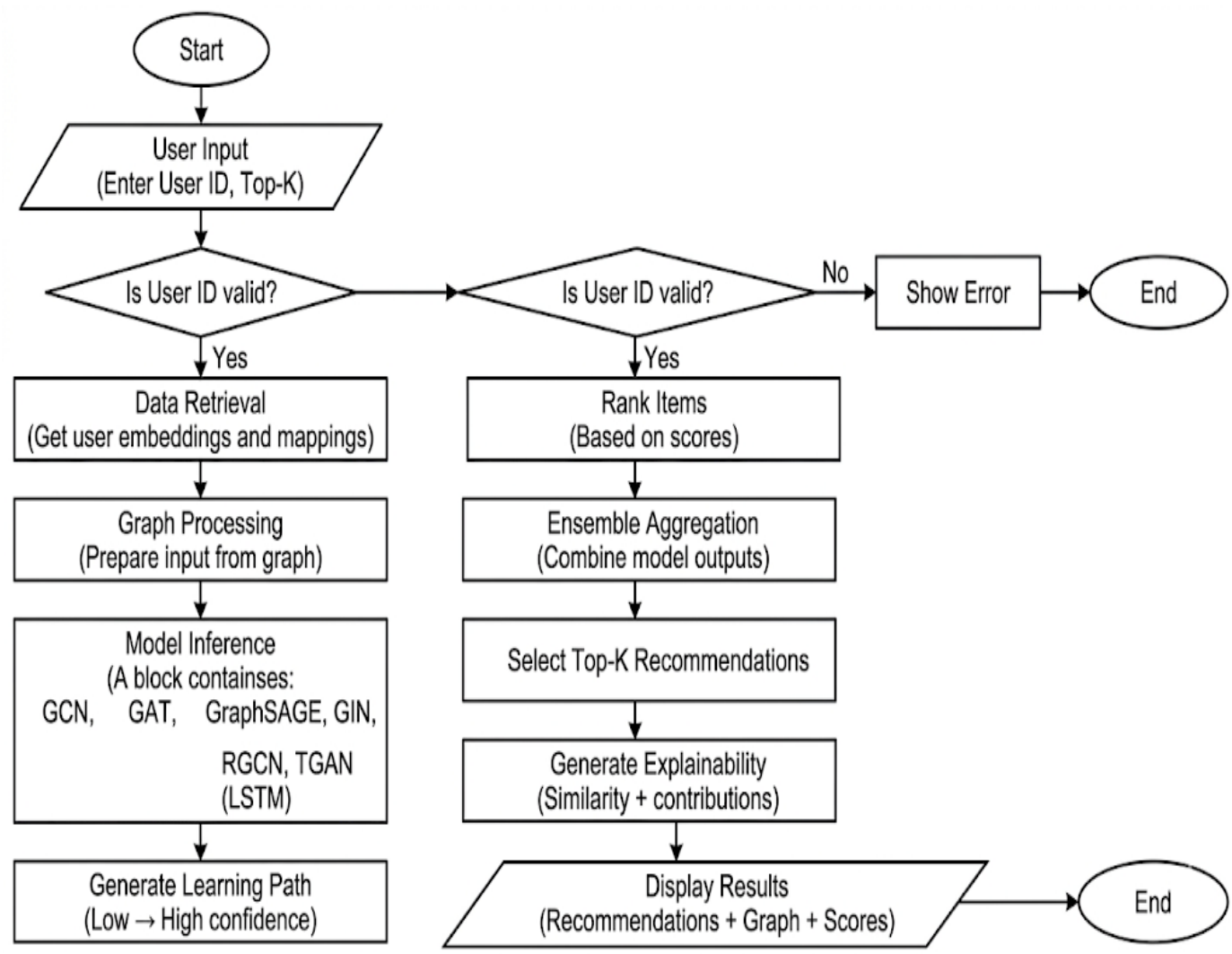
- **D1: Pre-trained Models**
Stores trained GNN models and TGAN

- **D2: Node Embeddings & Mappings**
Stores user, question, and skill mappings

- **D3: Interaction Dataset**
Stores processed user interaction data

The Data Flow Diagram illustrates how the system transforms user input into personalized recommendations through multiple stages of processing. It highlights the interaction between components and ensures clarity in understanding the system workflow. This structured flow enables efficient data handling, accurate predictions, and user-friendly output.

**6.3 Flowchart**




The flowchart represents the step-by-step workflow of the Learning Path Recommendation System, illustrating how user input is processed through different stages to generate personalized recommendations and structured learning paths. It provides a clear visual understanding of the system's operational logic and decision-making process.

**Overview of Workflow**

The system begins with user interaction and progresses through data processing, model inference, and result generation. Each step transforms the input data into meaningful outputs using graph-based learning and ensemble techniques.

**Step-by-Step Process**

**1. Start**

The process begins when the system is initialized and ready to accept user input.

**2. User Input**

The user enters a valid **user ID** and selects the number of recommendations (Top-K) through the Streamlit interface.

**3. Validate Input**

The system checks whether the entered user ID exists in the dataset.

- If valid → Proceed to next step
- If invalid → Display error message and stop process

**4. Data Retrieval**

The system retrieves relevant data associated with the user, including node mappings and embeddings.

**5. Graph-Based Processing**

The system prepares input data for graph-based models by extracting node embeddings and constructing required inputs.

**6. Model Inference**

The processed data is passed through multiple models:

- GNN Models (GCN, GAT, GraphSAGE, GIN, RGCN)
- Temporal Model (TGAN with LSTM)

Each model generates prediction scores independently.

**7. Ensemble Aggregation**

The system combines predictions from all models using an averaging method.

Final Score = Average of all model outputs

**8. Rank Items**

The system ranks all candidate questions based on their final scores.

**9. Select Top-K Recommendations**

The top-K highest scored questions are selected as recommendations.

**10. Generate Learning Path**

The selected recommendations are ordered into a structured learning path:

Low Confidence → High Confidence

This helps users progress step-by-step.

**11. Generate Explainability**

The system computes similarity scores and model contributions to provide insights into recommendations.

**12. Display Results**

The frontend displays:

- Recommended questions
- Prediction scores
- Learning path
- Knowledge graph visualization
- Explanation details

**13. End**

The process completes after displaying results to the user.

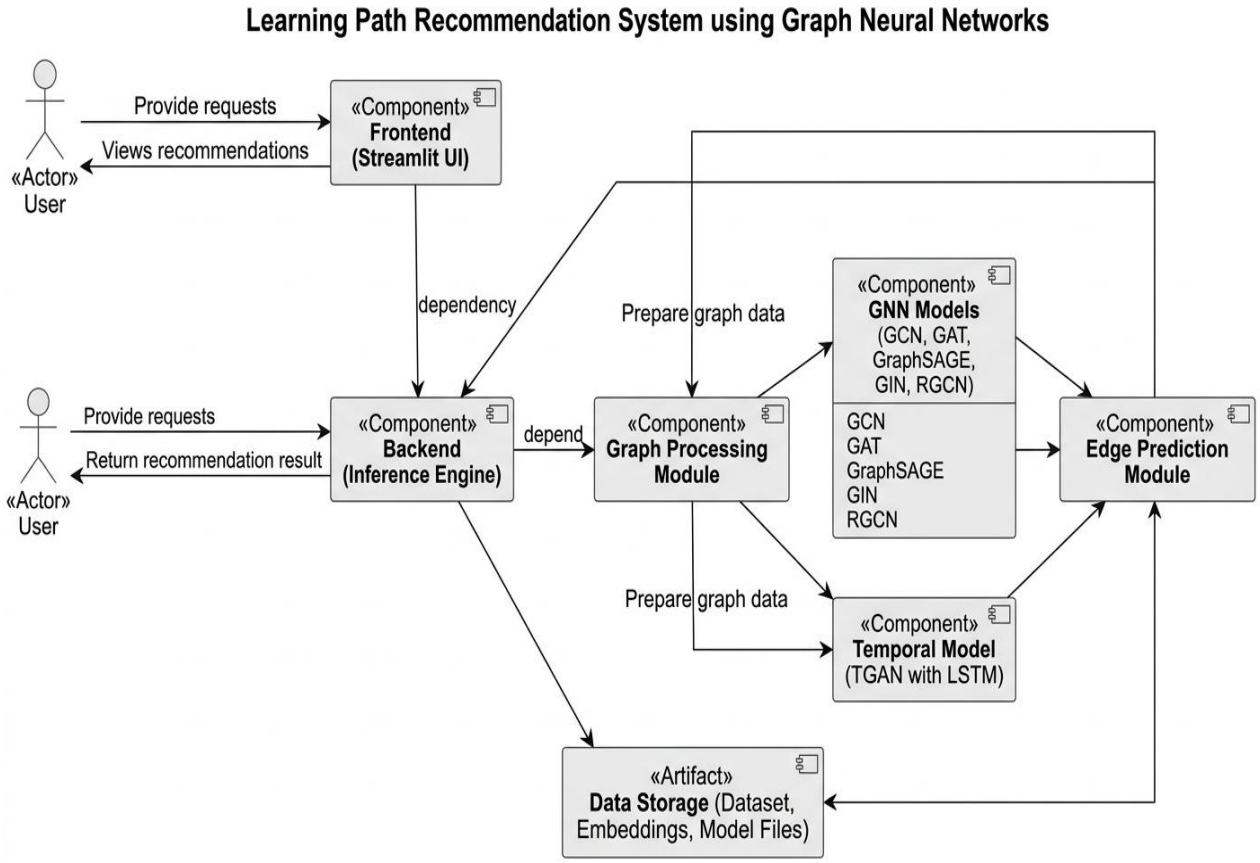
The flowchart clearly illustrates the sequential operation of the system, from user input to final output. It highlights how graph-based models and ensemble learning work together to produce accurate and structured recommendations. This workflow ensures efficiency, clarity, and improved user experience.

### 6.4 UML Diagrams

Unified Modeling Language (UML) diagrams are used to visually represent the structure and behavior of the Learning Path Recommendation System. These diagrams help in understanding system components, workflows, and interactions between users and the system. The following UML diagrams are used in this project:

- Component Diagram
- Activity Diagram
- State Machine Diagram
- Use Case Diagram

### 6.4.1 Component Diagram




The Component Diagram represents the overall structure of the system and shows how different components interact with each other.

**Main Components:**

- **Frontend (Streamlit UI):**
Handles user interaction, input, and visualization of results.

- **Backend (Inference Engine):**
Processes input data, loads models, and generates recommendations.

- **Graph Processing Module:**
Constructs graph and manages node embeddings.

- **GNN Models:**
Includes GCN, GAT, GraphSAGE, GIN, and RGCN.

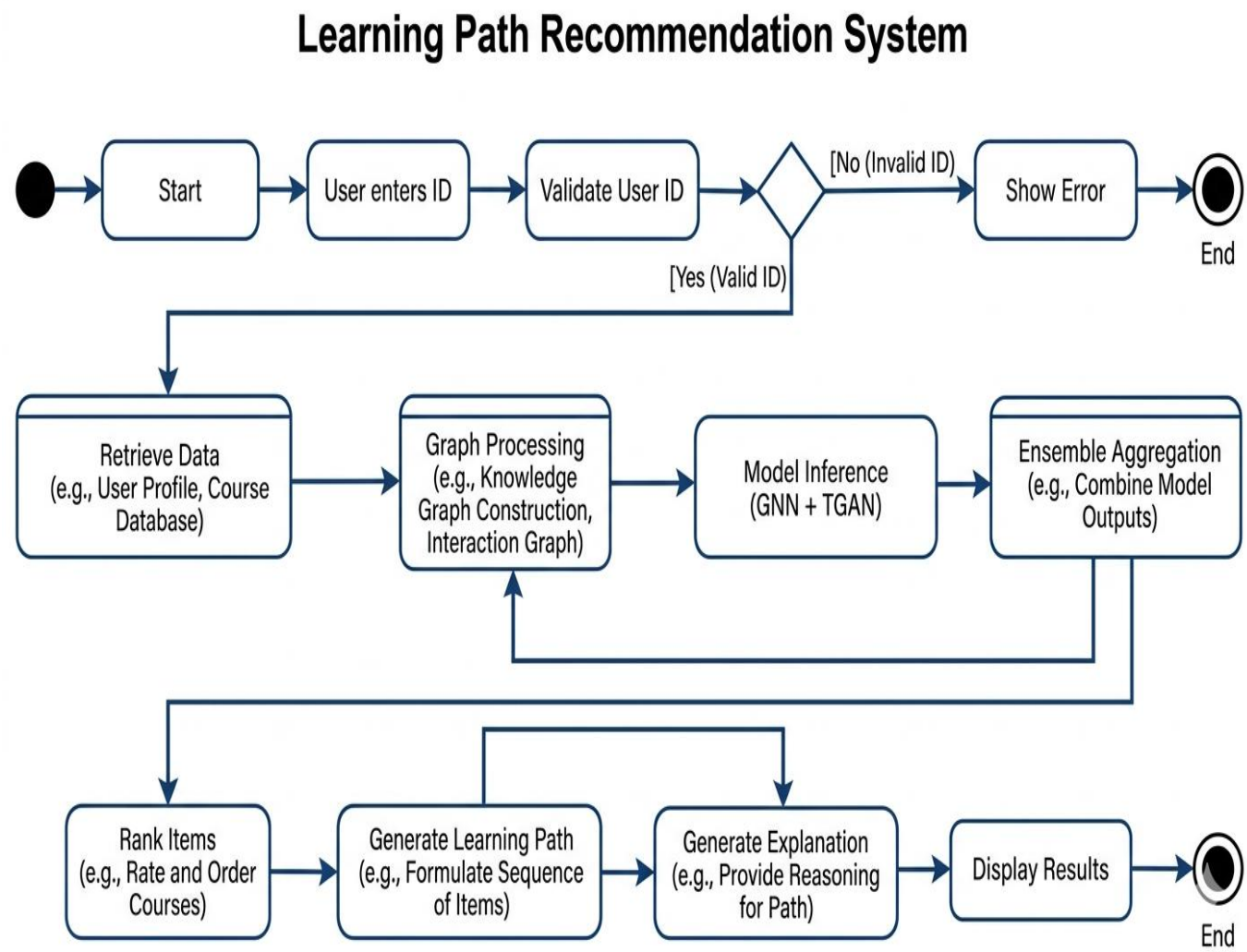
- **Temporal Model (TGAN):**
Captures sequential learning behavior.

- **Edge Prediction Module:**
Predicts interaction probability between user and items.

- **Data Storage:**
Stores dataset, embeddings, and trained models (.pth files).

This diagram shows how data flows from the user interface to backend processing and finally to the model layer.

### 6.4.2 Activity Diagram



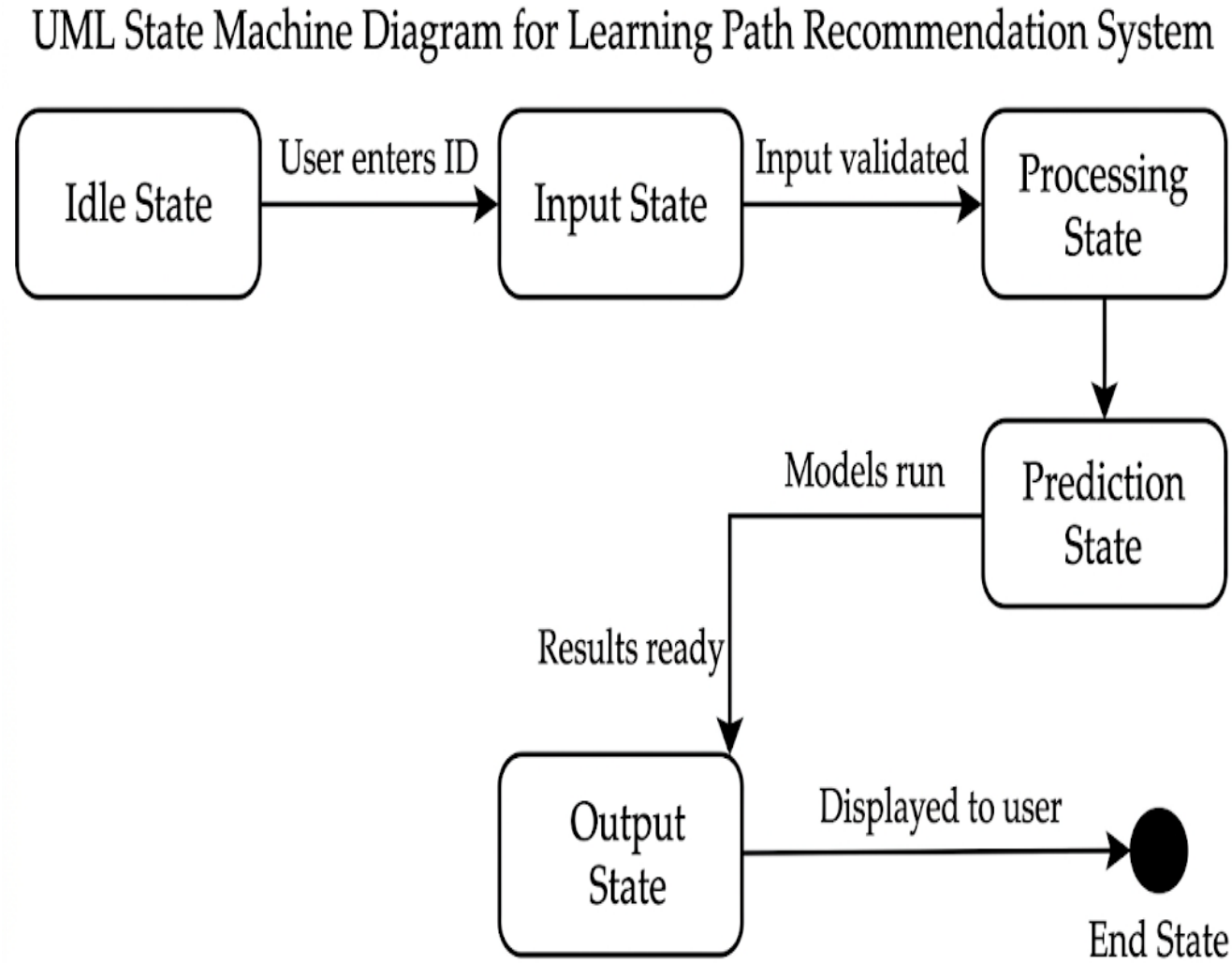

The Activity Diagram represents the workflow of the system, showing the sequence of operations from input to output.

**Flow Description:**

- User inputs ID
- System validates input
- Data is retrieved and processed
- Graph-based models generate predictions
- Ensemble combines results
- Recommendations are ranked
- Learning path is generated
- Results are displayed

This diagram highlights the logical flow of operations and decision points in the system.

### 6.4.3 State Machine Diagram



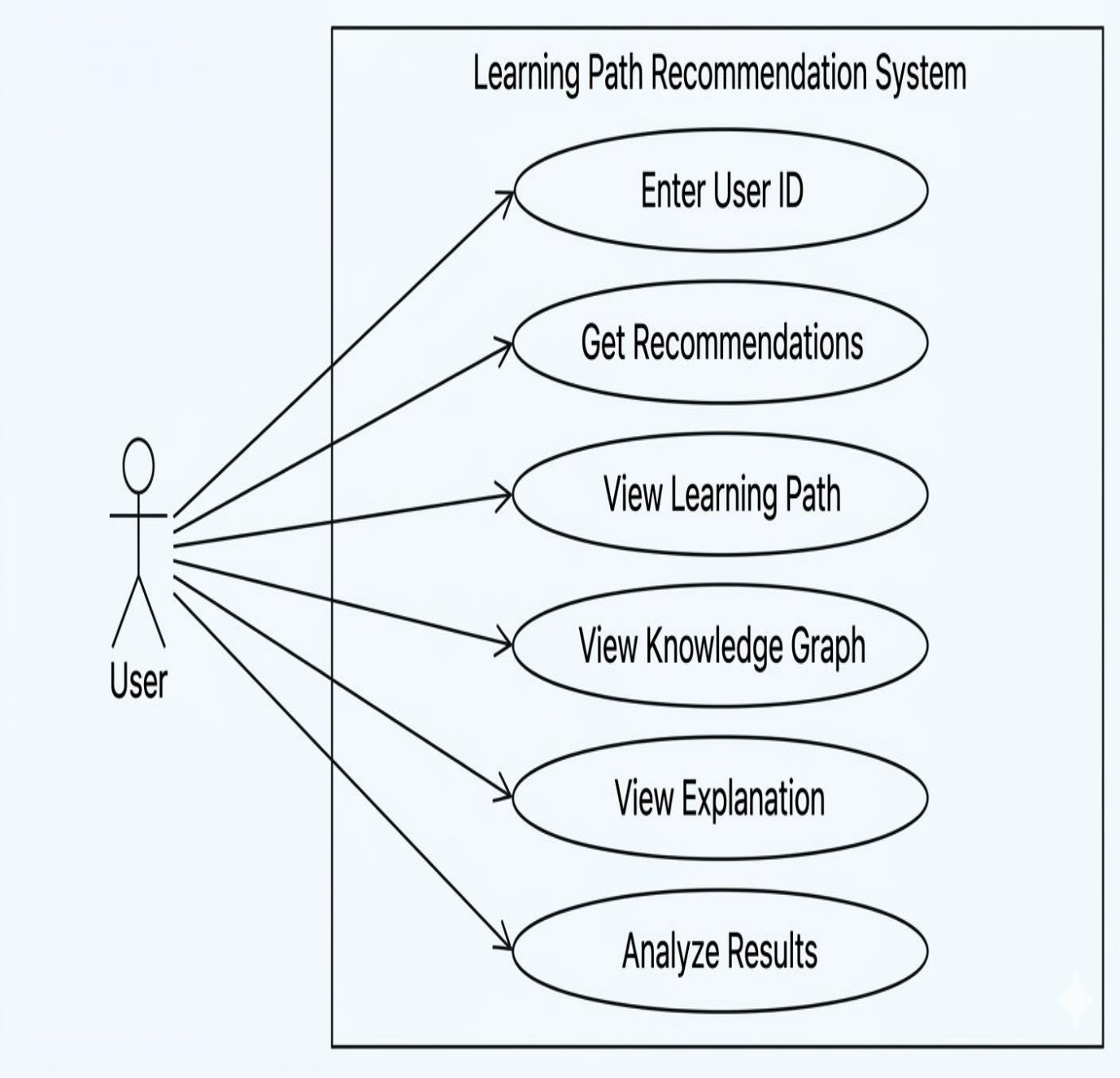

The State Machine Diagram represents different states of the system during execution.

**States:**

- Idle State – System waiting for input
- Input State – User provides input
- Processing State – Data preprocessing and model inference
- Prediction State – Generating recommendation scores
- Output State – Displaying results
- End State – Process completed

Transitions occur based on events such as input submission, processing completion, and result generation.

### 6.4.4 Use Case Diagram




The Use Case Diagram represents interactions between the user and the system.

**Actor:**
- User (Learner)

**Use Cases:**

- Enter User ID
- Request Recommendations
- View Learning Path
- View Knowledge Graph
- View Explanation
- Analyze Results

This diagram shows how the user interacts with different functionalities of the system.

The UML diagrams provide a clear understanding of both the structural and behavioral aspects of the system. They help in visualizing how components are organized, how data flows, and how users interact with the system. These diagrams play an important role in system design, documentation, and communication.

# CHAPTER – 7

# IMPLEMENTATION

## 7.1 System Setup and Implementation

The implementation of the Learning Path Recommendation System involves setting up the development environment, preparing the dataset, constructing the graph, training multiple models, and integrating the system into a functional application. This chapter describes the complete implementation process in a structured manner.

### 1. System Environment Setup

The system is implemented using Python due to its extensive support for machine learning and data processing libraries.

**Hardware Requirements:**
- Processor: Intel i5 or higher
- RAM: Minimum 8 GB
- Storage: At least 10 GB free space
- GPU (Optional): For faster model training

**Software Requirements:**
- Operating System: Windows / Linux / macOS
- Python Version: 3.10 or above

**Libraries Used:**
- PyTorch – Deep learning framework
- PyTorch Geometric – Graph Neural Network implementation
- NumPy – Numerical computations
- Pandas – Data manipulation
- Scikit-learn – Evaluation metrics
- Streamlit – Frontend interface
- Matplotlib / Plotly – Visualization

These tools provide a complete environment for developing and deploying the system.

### 2. Dataset Preparation

The system uses educational datasets such as EdNet, which contain user interaction records.

**Dataset Features:**
- User ID
- Question ID
- Timestamp
- User Answer / Correctness

**Preprocessing Steps:**
- Filtering relevant data
- Handling missing values

- Converting categorical data into numerical form
- Creating mappings for users, questions, and skills
- Limiting dataset size for efficient training

A new feature called **skill** is derived from question IDs to represent underlying concepts.

**3. Graph Construction**

The processed data is converted into a graph structure.

**Node Types:**
- Users
- Questions
- Skills

**Edge Types:**
- User → Question (interaction)
- Question → Skill (concept relationship)

**Implementation Steps:**
- Assign unique indices to nodes
- Create edge list using relationships
- Convert edges into tensor format
- Construct graph using PyTorch Geometric

This graph serves as the input for GNN models.

**4. Negative Sampling**

To train the model effectively, both positive and negative samples are required.

**Process:**
- Positive samples: Existing edges (real interactions)
- Negative samples: Randomly generated non-existing edges

This helps the model learn to distinguish between relevant and irrelevant recommendations.

**5. Model Implementation**

Multiple Graph Neural Network models are implemented:

- GCN (Graph Convolutional Network)
- GAT (Graph Attention Network)
- GraphSAGE
- GIN (Graph Isomorphism Network)
- RGCN (Relational GCN)

Each model takes node embeddings and graph structure as input and produces updated embeddings.

**6. Temporal Model (TGAN)**

A temporal model using LSTM is implemented to capture sequential learning behavior.

**Function:**
- Processes embeddings over time
- Captures learning progression
- Enhances recommendation accuracy

**7. Node Embeddings**

Each node (user, question, skill) is represented using an embedding vector.

**Implementation:**
- Initialize embedding layer
- Learn embeddings during training
- Store embeddings for inference

These embeddings capture relationships in the graph.

**8. Edge Prediction Module**

A neural network (MLP) is used to predict interaction probability.

**Steps:**
- Concatenate user and question embeddings
- Pass through multiple layers
- Apply activation functions
- Output probability score

This score determines recommendation relevance.

**9. Model Training**

The training process involves optimizing model parameters using training data.

**Key Steps:**
- Split data into training and testing sets
- Use Binary Cross-Entropy Loss
- Optimize using Adam optimizer
- Train for multiple epochs
- Monitor loss and performance

Multiple models are trained separately and later combined.

**10. Ensemble Learning**

Predictions from all models are combined.

**Method:**
- Average prediction scores from each model

This improves accuracy and stability of recommendations.

**11. Model Evaluation**

The system is evaluated using various metrics:

- Accuracy
- Precision
- Recall
- F1 Score
- AUC (Area Under Curve)
- NDCG@K

These metrics ensure both prediction quality and ranking performance.

**12. Model Saving and Loading**

After training, models are saved for later use.

**Saved Components:**
- Node embeddings
- Model weights
- Edge prediction module
- User and question mappings

Models are stored in .pth format and loaded during inference.

**13. Backend Implementation**

The backend is responsible for generating recommendations.

**Functions:**
- Load trained models
- Process user input
- Generate embeddings
- Compute scores
- Rank items

It ensures efficient execution of all computations.

**14. Frontend Implementation (Streamlit)****

A user-friendly interface is developed using Streamlit.

**Features:**
- Input user ID
- Select number of recommendations
- Display recommended questions
- Show learning path
- Visualize knowledge graph
- Display explanation

This makes the system interactive and easy to use.

**15. Knowledge Graph Visualization**

The system includes visualization of graph relationships.

**Tools Used:**
- PyVis / Network libraries

This helps users understand how recommendations are connected.

**16. Integration of System Components**

All components are integrated into a complete system:

- Data → Graph → Models → Predictions → UI

The frontend communicates with the backend to display results in real time.

The implementation of the system combines data processing, graph modeling, deep learning, and web-based visualization into a unified framework. By integrating multiple GNN models, temporal learning, and an interactive interface, the system provides accurate, explainable, and structured learning recommendations. The modular design ensures scalability and ease of maintenance, making it suitable for real-world applications.

# CHAPTER – 8

# RESULTS AND ANALYSIS

## 8.1 METHODOLOGY

The methodology adopted in this project focuses on designing a graph-based intelligent recommendation system that models learning behavior and predicts the most relevant learning items for each user.

*1. Data Preparation and Graph Construction*

The dataset (EdNet) consists of user interactions with questions over time. This data is preprocessed and transformed into a graph structure where:

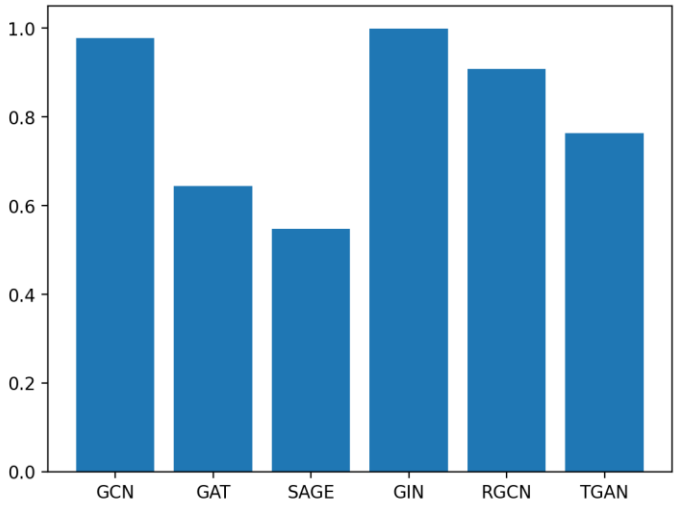
- Nodes represent Users, Questions, and Skills
- Edges represent interactions and concept relationships

This graph representation enables capturing both direct and indirect relationships among entities.

*2. Feature Representation*

Each node is represented using embedding vectors learned through Graph Neural Networks. These embeddings capture structural and contextual relationships within the graph.

*3. Model Architecture*




The system uses an ensemble of multiple GNN models:

- Graph Convolutional Network (GCN)
- Graph Attention Network (GAT)
- GraphSAGE
- Graph Isomorphism Network (GIN)
- Relational Graph Convolutional Network (R-GCN)

Each model captures different characteristics of the graph, improving overall performance.

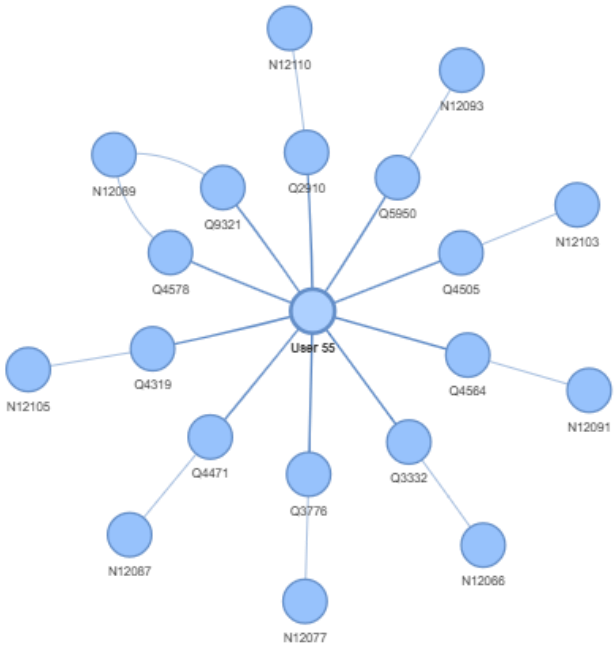
*4. Temporal Learning Component*

To model dynamic user behavior:

- Temporal Graph Attention Network (TGAN) is used
- Combined with LSTM to capture sequential learning patterns

This helps track how user knowledge evolves over time.

*5. Edge Prediction*




The recommendation problem is treated as an edge prediction task, where the model predicts the probability of interaction between users and questions. These probabilities are used as recommendation scores.

*6. Ensemble Strategy*

Outputs from all models are combined using an ensemble approach to improve accuracy, robustness, and generalization.

*7. Learning Path Generation*

Ordered from low confidence → high confidence

Step 1: Question q9022 (0.7383)

Step 2: Question q1515 (0.7402)

Step 3: Question q101 (0.7403)

Step 4: Question q157 (0.7409)

Step 5: Question q598 (0.7469)

Step 6: Question q303 (0.7521)

Step 7: Question q8120 (0.7546)

Step 8: Question q5 (0.7634)

Step 9: Question q3753 (0.7800)

Step 10: Question q176 (0.8226)

Recommended items are ordered from low confidence to high confidence to generate a structured learning path, guiding the learner step-by-step.

*8. Evaluation Metrics*

The system is evaluated using:
Accuracy, Precision, Recall, F1 Score, AUC, and NDCG.

## 8.2 RESULTS AND ANALYSIS

**Obtained Results**

Recommendations Learning Path Graph Metrics

ACCURACY
0.8272

PRECISION
0.8907

RECALL
0.7854

F1
0.8347

AUC
0.9117

NDCG
0.9917

- Accuracy: 82.72%
- Precision 89.07%
- Recall: 78.54%

- F1 Score: 83.47%
- AUC: 91.17%
- NDCG: 99.17%

**Result Interpretation**

**Accuracy (0.8272)**

Accuracy represents the overall correctness of predictions. A value of 82.72% indicates that the model performs well in predicting relevant interactions.

**Precision (0.8907)**

Precision measures how many recommended items are actually relevant. A high precision of 89.07% indicates that most recommendations are correct and useful.

**Recall (0.7854)**

Recall measures how many relevant items were successfully recommended. The value shows that the system captures a good portion of relevant items while maintaining quality.

**F1 Score (0.8347)**

F1 Score balances precision and recall. A value of 0.8347 indicates stable and reliable performance of the model.

**AUC (0.9117)**

AUC measures the model's ability to distinguish between relevant and irrelevant items. A value above 0.9 indicates excellent performance.

**NDCG (0.9917)**

NDCG evaluates ranking quality. A value close to 1 indicates that the most relevant recommendations are ranked at the top, making the system highly effective for learning path generation.

**Overall Analysis**

The results demonstrate that the proposed system:

- Produces highly accurate recommendations
- Maintains strong ranking performance
- Effectively balances precision and recall
- Generates structured and meaningful learning paths

The integration of graph-based modeling, temporal learning, and ensemble techniques significantly improves recommendation quality compared to traditional systems.

### 8.2 Learning Outcomes

The development of the Learning Path Recommendation System using Graph Neural Networks provided significant technical, analytical, and practical learning experiences. This project helped in understanding advanced machine learning concepts, system design principles, and real-world application development.

**Technical Learning Outcomes**

- **Understanding Graph Neural Networks (GNNs):**
Gained in-depth knowledge of graph-based learning techniques and how models such as GCN, GAT, GraphSAGE, GIN, and RGCN operate on graph-structured data.

- **Graph Data Modeling:**
Learned how to convert real-world interaction data into a graph format consisting of nodes and edges, enabling better representation of relationships.

- **Temporal Modeling:**
Understood how sequential models like LSTM (TGAN) capture time-based user behavior and improve prediction accuracy.

- **Ensemble Learning:**
Explored how combining multiple models enhances performance, reduces bias, and improves robustness.

- **Edge Prediction Techniques:**
Learned how to formulate recommendation problems as link prediction tasks and implement them using neural networks.

**Programming and Development Skills**

- **Python Programming:**
Improved proficiency in Python for data processing, model building, and system integration.

- **Machine Learning Libraries:**
Gained hands-on experience with PyTorch, PyTorch Geometric, Scikit-learn, and other libraries.

- **Data Handling:**
Learned how to preprocess large datasets using Pandas and NumPy.

- **Model Training and Evaluation:**
Understood training workflows, loss functions, optimization techniques, and evaluation metrics such as Accuracy, Precision, Recall, F1 Score, AUC, and NDCG.

- **Frontend Development:**
Developed an interactive user interface using Streamlit for real-time recommendation visualization.

**Analytical and Problem-Solving Skills**

- **Problem Identification:**
Understood limitations of traditional recommendation systems and identified the need for graph-based approaches.

- **Solution Design:**
Designed a system that integrates multiple models and techniques to address real-world challenges.

- **Performance Analysis:**
Analyzed model results using multiple metrics and interpreted their significance.

- **Optimization:**
Learned how to improve model performance through tuning, ensemble methods, and efficient data handling.

**Practical and Research Skills**

- **System Integration:**
Learned how to integrate backend machine learning models with a frontend application.

- **Experimentation:**
Conducted experiments with different models and compared their performance.

- **Understanding Research Concepts:**
Gained exposure to modern research areas such as graph learning, recommendation systems, and explainable AI.

- **Documentation:**
Developed skills in writing technical reports and presenting complex ideas in a structured manner.

**Soft Skills and Team Learning**

- **Project Planning:**
Managed different phases of the project, including design, implementation, and testing.

- **Critical Thinking:**
Analyzed problems and evaluated multiple approaches before selecting the best solution.

- **Communication Skills:**
Improved ability to explain technical concepts clearly during presentations and documentation.

The project provided a comprehensive learning experience that combines theoretical knowledge with practical implementation. It enhanced understanding of advanced machine learning techniques, system design, and real-world application development. The skills acquired through this project are valuable for future work in fields such as artificial intelligence, data science, and software development.

# CHAPTER – 9

# CONCLUSION

## 9.1 Conclusion

The rapid growth of digital learning platforms has significantly transformed the educational landscape, enabling learners to access a vast array of resources anytime and anywhere. These platforms provide diverse content such as practice questions, video lectures, tutorials, and interactive exercises, making learning more flexible and accessible than ever before. However, this abundance of content has also created a critical need for intelligent systems that can effectively guide learners through this information-rich environment. Without proper guidance, learners often struggle to identify what content is relevant to their current knowledge level and what they should learn next. Traditional recommendation systems, while useful to some extent, fail to capture the complex relationships between users, learning materials, and underlying concepts. They also lack adaptability, structured learning guidance, and explainability, which are essential for effective learning. This project addresses these challenges by proposing a Graph Neural Network-based Learning Path Recommendation System.

The primary objective of this project was to design and develop a system capable of generating personalized, accurate, and structured learning recommendations. This objective has been successfully achieved by modeling the learning environment as a graph consisting of users, questions, and skills. In this representation, nodes correspond to different entities such as learners, learning items, and conceptual topics, while edges represent interactions and relationships among them. By leveraging this graph-based representation, the system is able to capture both direct and indirect relationships, enabling a deeper and more comprehensive understanding of learning patterns. This approach overcomes the limitations of traditional flat data representations and allows the system to model the interconnected nature of knowledge more effectively.

A major strength of this project lies in the integration of multiple Graph Neural Network models, including Graph Convolutional Networks (GCN), Graph Attention Networks (GAT), GraphSAGE, Graph Isomorphism Networks (GIN), and Relational Graph Convolutional Networks (RGCN). Each of these models contributes unique capabilities in learning graph structures and relationships. For instance, GCN captures neighborhood information through convolution operations, GAT introduces attention mechanisms to weigh the importance of different neighbors, GraphSAGE enables inductive learning for unseen nodes, GIN provides strong discriminative power for graph structures, and RGCN handles heterogeneous relationships effectively. By combining these models using an ensemble approach, the system benefits from their complementary strengths. This results in improved accuracy, robustness, and generalization compared to relying on any single model. The ensemble framework also reduces the risk of overfitting and enhances the stability of predictions across different scenarios.

Another significant contribution of this project is the incorporation of temporal learning using a Temporal Graph Attention Network (TGAN) integrated with Long Short-Term Memory (LSTM). Learning is inherently a dynamic process, where a learner's knowledge evolves over time as they interact with educational content. Traditional recommendation systems often fail to capture this temporal dimension, treating all interactions as static and equally important. In contrast, the proposed system models the sequential nature of user interactions, allowing it to understand how a learner's knowledge state changes over time. The use of LSTM enables the system to capture long-term dependencies in learning behavior, while the temporal attention mechanism highlights the most relevant past interactions. This combination ensures that recommendations are adaptive, context-aware, and aligned with the learner's current level of understanding. By integrating both structural and temporal aspects of data, the system achieves a more holistic representation of the learning process.

The recommendation process in this system is formulated as an edge prediction problem, where the goal is to predict the likelihood of interaction between a user and a question. This formulation allows the model to assign meaningful prediction scores to potential interactions, which can then be used to rank items and generate personalized recommendations. Unlike traditional recommendation approaches that rely on heuristic methods, this probabilistic framework provides a more principled and data-driven way of generating recommendations. The predicted scores reflect the relevance and suitability of each item for a given user, enabling precise and targeted recommendations.

Furthermore, the system extends beyond simple recommendation lists by organizing the suggested items into a structured learning path. Learning is not a random process but a sequential progression from basic to advanced concepts. Recognizing this, the system arranges recommended questions in a logical order, typically progressing from lower confidence items to higher confidence ones. This structured approach helps learners build foundational

knowledge before moving on to more complex topics. It also reduces cognitive overload and improves learning efficiency by providing a clear roadmap for progression. The concept of learning paths significantly enhances the practical usefulness of the system and aligns it with real-world educational needs.

A key highlight of the proposed system is its emphasis on explainability. Many modern deep learning models operate as black boxes, providing little to no insight into how predictions are made. This lack of transparency can reduce user trust and limit the adoption of such systems in educational settings. In contrast, this project incorporates explainability features that provide insights into the reasoning behind recommendations. Techniques such as similarity measures and model contribution analysis are used to explain why certain items are recommended. These explanations help users understand the underlying logic of the system, making the recommendations more interpretable and trustworthy. Explainability also enables educators to validate the system's behavior and make informed decisions based on its outputs.

The system has been successfully implemented as a web-based application using Streamlit, which provides an interactive and user-friendly interface. The frontend allows users to easily input their user ID and generate personalized recommendations. It also enables users to explore recommended questions, view structured learning paths, visualize knowledge graphs, and understand the reasoning behind the recommendations. The integration of backend machine learning models with frontend visualization ensures a seamless end-to-end experience. This makes the system accessible to users without requiring technical expertise, thereby increasing its usability and adoption potential.

From an experimental perspective, the system demonstrates strong performance across multiple evaluation metrics, including Accuracy, Precision, Recall, F1 Score, Area Under the Curve (AUC), and Normalized Discounted Cumulative Gain (NDCG). These metrics evaluate different aspects of the system, such as prediction correctness, ranking quality, and overall effectiveness. The high values obtained across these metrics indicate that the system performs well in both classification and ranking tasks. The results also validate the effectiveness of the proposed approach, particularly the use of graph-based learning, ensemble techniques, and temporal modeling. Comparative analysis shows that the proposed system outperforms traditional recommendation methods, highlighting its superiority in capturing complex learning patterns.

In addition to its technical achievements, this project contributes to the broader field of educational technology by demonstrating how advanced machine learning techniques can be applied to enhance personalized learning experiences. The system addresses several real-world challenges, including data sparsity, lack of structured guidance, and absence of explainability. By providing a comprehensive solution that integrates multiple advanced techniques, the project sets a benchmark for future research in learning recommendation systems. It also highlights the potential of Graph Neural Networks in modeling complex relationships in educational data.

Despite its many strengths, there are opportunities for further improvement and extension. One possible enhancement is the incorporation of real-time data updates, which would allow the system to continuously adapt to new user interactions. This would further improve the responsiveness and accuracy of recommendations. Another area for improvement is the modeling of skill hierarchies, where relationships between concepts can be explicitly represented to provide more accurate learning paths. Additionally, scaling the system to handle larger datasets and more complex graphs would enhance its applicability in large-scale educational platforms.

Future work may also focus on improving explainability through advanced attention-based mechanisms, which can provide more detailed insights into model decisions. Deploying the system as a full-scale production application with robust backend infrastructure and user management features is another important direction. Integration with existing learning management systems (LMS) could further increase its practical impact and usability.

Overall, this project successfully demonstrates the design and implementation of an intelligent, scalable, and explainable learning path recommendation system. By combining Graph Neural Networks, temporal learning, and ensemble methods, the system provides a powerful and innovative solution that aligns with the needs of modern learners. It effectively addresses the limitations of traditional recommendation systems and offers a structured, adaptive, and user-centric approach to learning.

The project not only achieves its primary objectives but also opens new avenues for research and development in personalized education. It highlights the importance of leveraging advanced technologies to create meaningful and impactful learning experiences. As digital education continues to evolve, systems like the one proposed in this project will play a crucial role in shaping the future of learning by making it more intelligent, efficient, and accessible.

In conclusion, the proposed Learning Path Recommendation System represents a significant advancement in the field of educational technology. By transforming unstructured recommendation methods into a structured and explainable framework, it provides a practical and effective solution to the challenges faced by modern learners. The successful implementation and strong performance of the system demonstrate its potential for real-world deployment. This project serves as a foundation for future innovations and reinforces the value of graph-based approaches in building next-generation recommendation systems.

# CHAPTER – 10

# REFERENCES

## 10.1 References

The following references were used for understanding the concepts of recommendation systems, graph neural networks, temporal modeling, and system implementation:


1. Kipf, T. N., & Welling, M. (2017).
   "Semi-Supervised Classification with Graph Convolutional Networks."
   International Conference on Learning Representations (ICLR).

2. Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2018).
   "Graph Attention Networks."
   International Conference on Learning Representations (ICLR).

3. Hamilton, W., Ying, R., & Leskovec, J. (2017).
   "Inductive Representation Learning on Large Graphs (GraphSAGE)."
   Advances in Neural Information Processing Systems (NeurIPS).

4. Xu, K., Hu, W., Leskovec, J., & Jegelka, S. (2019).
   "How Powerful are Graph Neural Networks?"
   International Conference on Learning Representations (ICLR).

5. Schlichtkrull, M., Kipf, T. N., Bloem, P., van den Berg, R., Titov, I., & Welling, M. (2018).

"Modeling Relational Data with Graph Convolutional Networks (RGCN)."
European Semantic Web Conference (ESWC).

6. Rossi, E., Chamberlain, B. P., Frasca, F., Eynard, D., Monti, F., & Bronstein, M. (2020).
"Temporal Graph Networks for Deep Learning on Dynamic Graphs."

7. Hochreiter, S., & Schmidhuber, J. (1997).
"Long Short-Term Memory."
Neural Computation Journal.

8. Koren, Y., Bell, R., & Volinsky, C. (2009).
"Matrix Factorization Techniques for Recommender Systems."
IEEE Computer Society.

9. Ricci, F., Rokach, L., & Shapira, B. (2015).
"Recommender Systems Handbook."
Springer Publications.

10. Piech, C., et al. (2015).
"Deep Knowledge Tracing."
Advances in Neural Information Processing Systems (NeurIPS).

11. EdNet Dataset (KT1, KT3).
Available at: https://github.com/riiid/ednet

12. PyTorch Documentation.
https://pytorch.org/docs

13. PyTorch Geometric Documentation.
https://pytorch-geometric.readthedocs.io

14. Scikit-learn Documentation.
https://scikit-learn.org

15. Streamlit Documentation.
https://docs.streamlit.io

16. Ying, R., He, R., Chen, K., Eksombatchai, P., Hamilton, W. L., & Leskovec, J. (2018).
"Graph Convolutional Neural Networks for Web-Scale Recommender Systems."
ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD).
17. Wu, L., Sun, P., Fu, Y., Hong, R., Wang, X., & Wang, M. (2020).
"A Comprehensive Survey on Graph Neural Networks."
IEEE Transactions on Neural Networks and Learning Systems.
18. Zhang, S., Yao, L., Sun, A., & Tay, Y. (2019).
"Deep Learning Based Recommender System: A Survey and New Perspectives."
ACM Computing Surveys.
19. Berg, R. van den, Kipf, T. N., & Welling, M. (2017).
"Graph Convolutional Matrix Completion."
arXiv preprint arXiv:1706.02263.
20. Wang, X., He, X., Wang, M., Feng, F., & Chua, T. S. (2019).
"Neural Graph Collaborative Filtering."
ACM SIGIR Conference on Research and Development in Information Retrieval.
21. He, X., Deng, K., Wang, X., Li, Y., Zhang, Y., & Wang, M. (2017).
"LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation."
ACM SIGIR.

22. Sun, F., Hoffmann, J., & Tang, J. (2018).
"neural Collaborative Filtering with Graph Neural Networks."
arXiv preprint arXiv:1810.00826.
23. Zhou, J., Cui, G., Hu, S., et al. (2020).
"Graph Neural Networks: A Review of Methods and Applications."
AI Open Journal.
24. Velickovic, P. (2021).
"Graph Representation Learning."
Cambridge University Press.
25. Monti, F., Bronstein, M., & Bresson, X. (2017).
"Geometric Deep Learning on Graphs and Manifolds."
IEEE Signal Processing Magazine.
26. Grover, A., & Leskovec, J. (2016).
"node2vec: Scalable Feature Learning for Networks."
ACM SIGKDD.
27. Perozzi, B., Al-Rfou, R., & Skiena, S. (2014).
"DeepWalk: Online Learning of Social Representations."
ACM SIGKDD.
28. Rendle, S. (2010).
"Factorization Machines."
IEEE International Conference on Data Mining (ICDM).
29. Covington, P., Adams, J., & Sargin, E. (2016).
"Deep Neural Networks for YouTube Recommendations."
ACM RecSys.
30. Hidasi, B., Karatzoglou, A., Baltrunas, L., & Tikk, D. (2016).
"Session-Based Recommendations with Recurrent Neural Networks."
International Conference on Learning Representations (ICLR).
31. Wang, H., Zhang, F., Hou, M., Xie, X., Guo, M., & Liu, Q. (2018).
"SHINE: Signed Heterogeneous Information Network Embedding for Sentiment Link Prediction."
ACM WSDM.
32. Chen, J., Ma, T., & Xiao, C. (2018).
"FastGCN: Fast Learning with Graph Convolutional Networks via Importance Sampling."
International Conference on Learning Representations (ICLR).
33. Kipf, T. N., & Welling, M. (2016).
"Variational Graph Auto-Encoders."
arXiv preprint arXiv:1611.07308.
34. Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., & Yu, P. S. (2020).
"A Comprehensive Survey on Graph Neural Networks."
IEEE Transactions on Neural Networks.
35. Hamilton, W. L. (2020).
"Graph Representation Learning."
Morgan & Claypool Publishers.